AUTOFORGE: A CHECKPOINTED, HUMAN–IN–THE–LOOP MULTI-AGENT SDLC PIPELINE

COMMON INTEGRATED SOLUTION REPORT

PROJECT ID: R26-SE-033

Team member	Student ID	Individual component
Santhuka D.N.M.D	IT22098078	Requirement Agent & Domain Agent
Sansala T.G.B. D	IT22174826	Architecture Agent & UI/UX Agent
Dissanayake T.C	IT22157232	Coder Agent
Oberathna R.D.T. D	IT22102478	Security Agent & QA Agent

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) Degree in Information Technology

Specialized in Software Engineering

Department of Computing

Sri Lanka Institute of Information Technology

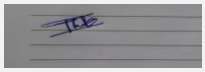
Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. We declare that this common report is our own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of our knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. We grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this report, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. We have disclosed all use of generative AI and other automated tools in accordance with the module policy. We remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this report.

Student name	Student ID	Signature	Date
Santhuka D.N.M.D	IT22098078		05/10/2026
Sansala T.G.B.D	IT22174826		05/10/2026
Dissanayake T.C	IT22157232		05/10/2026
Oberathna R.D.T. D	IT22102478		05/10/2026

Supervisor certification. The above candidates carried out this research project under my supervision.

Role and name	Signature	Date
Supervisor: **Prof. Nuwan Kodagoda**		
Co-supervisor: **Mr. Eishan Weerasinghe**		10/10/2026

ABSTRACT

Large language models can write a working function or a page on request, but a complete web application depends on a chain of decisions: what to build, how to structure it, how it should look, how to implement it, and whether the result is safe and correct. Tools that jump from one prompt to a finished application skip that chain, leaving nothing for a person to check. Existing multi-agent frameworks divide the work into roles, yet review happens through agent dialogue or after the run ends, and an interrupted run is usually lost. This report presents AutoForge, an integrated system in which seven agents (Requirement, Domain, Architecture, UI/UX, Coder, Security and Quality Assurance) carry an e-commerce feature from a rough requirement to tested code. The work is organised into four components, one per team member, joined by a LangGraph state graph that pauses for a human decision after every stage and checkpoints to MongoDB, so an approval given days later, even after a server restart, resumes the pipeline exactly where it stopped. Following a design-science approach, the system was evaluated on three features of increasing complexity: Item Listing, Authentication/Authorization and Shopping Cart. Every stage produced a versioned artefact with a recorded human decision, and four of the seven stages needed a revision on the first feature. With human gates switched on, the QA pass rate rose from 78% to 91% and escaped defects fell from five to one, at a cost of 36 minutes of review effort. Retrieval-based enrichment reduced missing requirements from seven to two, the Security Agent reached 87.5% precision and 77.8% recall on injected vulnerabilities, test coverage reached 79%, and all seven controlled interruptions recovered correctly. The results show that a checkpointed, human-approved pipeline is a practical and auditable way to use LLMs in software development.

Keywords: multi-agent systems, human-in-the-loop software engineering, large language models, checkpointed orchestration, software quality assurance

ACKNOWLEDGEMENTS

We are grateful to our supervisor, Prof.Nuwan Kodagoda, and our co-supervisor, Mr. Eishan Weerasinghe, for their guidance throughout this project. Their questions at each milestone pushed us to back our claims with measurements rather than descriptions, and that habit shaped how the evaluation in this report was designed.

We also thank the lecturers and staff of the Department of Computing at the Sri Lanka Institute of Information Technology for the teaching, facilities and feedback that made the work possible, and the panel members whose comments at the progress presentations helped us sharpen the scope of the research.

We acknowledge the maintainers of the open-source projects AutoForge is built on, including LangGraph, ChromaDB, Ollama, FastAPI, Playwright and PlantUML. Without their freely available work, a student team could not have built and tested a system of this size.

Finally, we thank our families and friends for their patience and encouragement during the many long evenings this project required.

TABLE OF CONTENTS

1. INTRODUCTION AND PROJECT CONTEXT 1
1.1 Background and context 1
1.1.1 From single prompts to multi-agent pipelines 2
1.1.2 Coding agents that start from a specified task 2
1.1.3 Human oversight in agent-based development 3
1.1.4 Retrieval, requirements and traceability 3
1.1.5 Verifying generated code 4
1.2 Stakeholders and user needs 4
1.3 Alignment with the Sustainable Development Goals 6
1.4 Purpose and scope of the common report 6
2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS 8
2.1 Overall problem definition 8
2.2 Root causes and overall research gap 8
2.3 Subproblem and component mapping 10
2.4 Overall aim and group objectives 11
2.5 Scope assumptions and constraints 12
3 OVERALL SOLUTION STRUCTURE 13
3.1 Integrated solution overview 13
3.2 High-level architecture 14
3.3 Component 1: Requirement Generation & Domain Enrichment 15
3.4 Component 2: Architecture Planning and UI/UX Design Generation 15
3.5 Component 3: Code Generation with Sandboxed Verification 16
3.6 Component 4: Security Analysis and Quality Assurance 17
3.7 End to end user and data flow 18

4 COMPONENT INTEGRATION AND COHESIVE OPERATION 19
4.1 Integration strategy 19
4.2 Interface data contracts and shared models 20
4.3 Shared nonfunctional requirements 22
4.4 Integrated workflow and exception handling 23
4.5 Team contribution and dependency matrix 24
5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION 26
5.1 Common technology stack and infrastructure 26
5.2 Development and integration process 27
5.3 Deployment and operational configuration 28
5.4 Evidence of team execution 28
6 INTEGRATED TESTING EVALUATION AND VALIDATION 30
6.1 Evaluation framework and success criteria 30
6.2 Integration and end to end testing 31
6.3 Overall results 32
6.3.1 Item Listing (CRUD) case study 32
6.3.2 Multi-feature evaluation 33
6.3.3 Domain retrieval (RAG) ablation 34
6.3.4 Security detection 35
6.3.5 QA coverage 35
6.3.6 Human-gates ablation 36
6.3.7 Checkpoint recovery 37
6.3.8 Execution performance 37
6.4 Stakeholder and user validation 38
6.5 Achievement of common objectives 39
6.6 Integrated limitations and threats 40

7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION 42
7.1 Value proposition and beneficiaries 42
7.2 Market alternatives and differentiation 44
7.3 Business model and feasibility 47
7.4 Adoption intellectual property and compliance 52
7.5 Sustainable Development Goal contribution 54
8 CONCLUSION AND FUTURE DIRECTIONS 55
8.1 Overall conclusion 55
8.2 Principal integrated contribution 55
8.3 Future directions 56
REFERENCES 57
APPENDICES 60
Appendix A Integrated interface specification 60
A.1 Stage gate interface 60
A.2 Shared sandbox interface 60
Appendix B Team contribution declaration 61
Appendix C Microsoft Teams Planner 61
Appendix D User Evidence 62

LIST OF FIGURES

Figure 3.1: AutoForge system diagram 14
Figure 4.1: Integrated workflow at every stage 23
Figure 6.1: Specification completeness with retrieval enabled and disabled 34
Figure 6.2: Quality outcomes with human gates switched on & off 36
Figure 6.3: Execution time and token usage per run 38
Figure 7.1: Monthly profit of the hosted service by number of Team seats 51
Figure C.1: Microsoft Teams Planner 62
Figure D.1: Overall technical feedback on the AutoForge system 63

LIST OF TABLES

Table 1.1:Stakeholders and their needs 5
Table 2.1: Comparison with related multi-agent software engineering systems 9
Table 2.2: Subproblems mapped to components 10
Table 2.3: Group objectives, responsible components and evaluation evidence 11
Table 3.1: Key functional requirements of the integrated solution 13
Table 3.2: Summary of the four components 17
Table 4.1: Artefact contracts between stages 20
Table 4.2: External interfaces and their failure handling 21
Table 4.3: Shared non-functional requirements and evidence 22
Table 4.4: Failure paths and recovery behaviour 24
Table 4.5: Team contribution and dependency matrix 25
Table 5.1: Common technology stack and justification 26
Table 6.1: Research questions, metrics and success criteria 30
Table 6.2: Integrated test scenarios 31
Table 6.3: Item Listing (CRUD): final artefact version &approval 32
Table 6.4: Multi-feature complexity evaluation 33
Table 6.5: Domain retrieval (RAG) ablation 34
Table 6.6: Security detection metrics 35
Table 6.7: QA coverage by test category 35
Table 6.8: Human gates ON versus OFF 36
Table 6.9: Controlled checkpoint recovery trials 37
Table 6.10: Execution performance 38
Table 6.11: Achievement of group objectives 40
Table 7.1: Value created for each beneficiary 42
Table 7.2: Stakeholder evidence of demand 44
Table 7.3: Pricing existing AI development tools (October 2026) 45
Table 7.4: Alternatives and AutoForge's differentiation 46
Table 7.5: Proposed AutoForge packages 47

Table 7.6: Business model summary 49
Table 7.7: One-time development investment 49
Table 7.8: Validation and launch plan 52
Table A.1: Stage gate interface 60
Table B.1: Individual and shared contributions 61

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
FN	False Negative
FP	False Positive
FR	Functional Requirement
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
LLM	Large Language Model
NFR	Non-Functional Requirement
PNG	Portable Network Graphics
QA	Quality Assurance
RAG	Retrieval-Augmented Generation
RQ	Research Question
SDG	Sustainable Development Goal
SDLC	Software Development Life Cycle
SRS	Software Requirements Specification
TP	True Positive
UI	User Interface
UML	Unified Modelling Language
UX	User Experience

1. INTRODUCTION AND PROJECT CONTEXT

1.1 Background and context

A large language model (LLM) can already write a working function, an interface page or a single file when asked. A full web application is a different kind of task. It means working through a chain of decisions: what should be built, how the system should be structured, how it should look, how it should be implemented, and whether the finished result is safe and correct. In a normal software team, each of these decisions leaves something concrete behind, such as a requirements document, a set of design diagrams, screen mock-ups, a code change or a test report. Those intermediate documents are what let people notice a mistake early, argue about it, and fix it before it spreads.

Tools that try to generate a complete application from one prompt skip most of that chain. The user gets code at the end, but there is no specification to check and no design to question. When the result is poor, it is hard to tell whether the model misunderstood the requirement or simply made a coding mistake, because nothing between the prompt and the code was ever written down for a person to read.

This matters a great deal in e-commerce, the domain this project targets. An online store handles user accounts, product data, stock levels, carts, orders and payment details. A wrong early assumption, for example about how inventory or payment information should be modelled, costs very little to correct while the specification is still being reviewed, but becomes expensive once code, database models and screens have been built on top of it. E-commerce is also a domain with well-known conventions (product catalogues, administrator roles, order histories) that a general-purpose model does not always apply unless it is reminded of them.

This section reviews the previous work that our project builds upon. It first discusses multi-agent systems that generate software by assigning different tasks to cooperating agents. It then looks at coding agents that work from already-defined software requirements. Next, it examines research on human oversight and interaction with AI systems. Finally, it discusses studies on retrieval, traceability, and

automated verification, which influenced the design of the individual agents in our system.

1.1.1 From single prompts to multi-agent pipelines

Hou et al. [1] reviewed a large body of research on LLMs for software engineering and found that most systems automate one phase of the software development life cycle (SDLC), such as code generation or test generation, rather than coordinating several phases with durable oversight at each step. That observation is the starting point for our work.

Two systems moved furthest towards covering more than one phase. ChatDev [2] organises role-playing agents into a chat-based "virtual software company" that follows a waterfall-like process, with agents acting as designers, programmers and testers who talk to one another. MetaGPT [3] takes a more structured route: its role agents exchange documents, including a specification like a Software Requirements Specification (SRS) and a design document, instead of free-form conversation. Both are closer to how a real engineering team works than a single prompt is. However, in both systems, review happens through further agent dialogue or only after the whole run has finished. Neither exposes a persistent checkpoint at which a person can approve or reject the output of a particular stage.

He, Treude and Lo [4] surveyed LLM-based multi-agent systems for software engineering and set out a research agenda for the field. Two of the open problems they name, human-agent collaboration and the verifiability of results, are the problems AutoForge is designed around.

1.1.2 Coding agents that start from a specified task

A second group of systems assumes that the task is already specified, usually as an issue in a GitHub repository. SWE-agent [5] introduced an agent-computer interface that lets a model navigate a repository, edit files and run commands to resolve such an issue. OpenHands [6] generalizes this idea into an open platform where agents interact with a software environment. CodeAct [7] has agents express their actions as executable code, which is close to how AutoForge's Coder Agent works. HyperAgent [8] scales these ideas with a modular, generalist design, while Agentless

[9] shows that a much simpler, fixed pipeline can match elaborate agents on the SWE-bench benchmark [10].

These systems are impressive at repairing existing code, but their starting point is different from ours. They begin with a repository and an issue description. AutoForge begins with a rough business requirement and has to produce the specification, the design and the repository itself. Two other systems influenced specific agents. AgentCoder [11] uses a loop of programmer, test-designer and test-executor agents, which is the closest published design to our QA Agent. RepoCoder [12] improves code completion by retrieving code from the same repository; our Coder Agent does not do this yet, and we return to it as future work in Chapter 8.

1.1.3 Human oversight in agent-based development

In most of the systems above, "human in the loop" means that a person reads the final diff or the finished output after the fact. Takerngsaksiri et al. [13] took a different approach with HULA, a human-in-the-loop framework deployed inside Atlassian JIRA. An important finding from their deployment is that engineers valued oversight at the planning stage as much as oversight at the code stage. In other words, people wanted to check the plan before any code was written, not only the code once it existed. This result directly supports our decision to place an approval gate after every stage of the pipeline, including the early, non-code stages.

1.1.4 Retrieval, requirements and traceability

Role-based systems such as MetaGPT build their specification from the model's own parametric knowledge, meaning whatever the model happened to learn during training. Lewis et al. [14] introduced retrieval-augmented generation (RAG), in which the model first retrieves relevant passages from an external knowledge base and then generates its answer with those passages in view. A specification stage that consults a retrieved knowledge base should catch gaps earlier than one that relies on the model alone. Hemmat et al. [15], in a systematic review of LLMs in requirements engineering, describe this kind of domain grounding as an under-explored research direction.

Traceability is the other half of the requirements problem. Gotel and Finkelstein [16] described the requirements traceability problem more than three decades ago: the difficulty of following a requirement forwards into design and code, and backwards to where it came from. A recent systematic review by Mucha, Kaufmann and Riehle [17] reports that most traceability solutions still address one narrow type of link rather than a full chain from requirement to code. None of the multi-agent systems we reviewed reported versioned, addressable identifiers for the artefact produced at each stage, which is what a full chain would need.

1.1.5 Verifying generated code

Generated code is only useful if someone can trust it. Wang et al. [18] surveyed the use of LLMs in software testing, and Zhu et al. [19] surveyed the meeting point of LLMs and software security. Both surveys report that most systems lack a deterministic verification layer: the model is often asked whether the code is correct or safe, and its answer is taken at face value. Zhu et al. recommend combining conventional static analysis with LLM review rather than relying on either one alone. AutoForge's Security and QA agents follow that advice by running real scanners and a real test runner and treating any LLM opinion as an optional second read.

Taken together, this literature shows plenty of progress on individual phases, but no system that treats each SDLC stage as a separately reviewable, versioned artefact, survives interruption, grounds its specification in retrieved domain knowledge, and verifies its output with executable checks. Chapter 2 turns this observation into a precise problem statement and research gap.

1.2 Stakeholders and user needs

AutoForge is a tool for the people who build web applications, and indirectly for the people who use those applications. We identified the stakeholders below from the problem itself and from the roles the pipeline needs at each approval gate. The system diagram in Chapter 3 shows one of these roles explicitly: a business analyst who works with the Requirement Agent on the initial requirements.

The primary stakeholders are the developers and small development teams who would run AutoForge to build an e-commerce feature, and the reviewers who

approve each stage. In a larger organization these review roles would be held by different people: a business analyst for the requirements, a solution architect or technical lead for the architecture and the code, a designer for the interface, and a security or QA engineer for the final two stages. In a small team, one or two people may play all of them. The secondary stakeholders are the store owners who commission the application, the customers whose accounts and orders the application will hold, and the academic community studying human-agent collaboration in software engineering.

Table 1.1:Stakeholders and their needs

Stakeholder	Role in relation to AutoForge	Main needs
Developers and small teams	Run the pipeline to build features	See and steer each intermediate decision; avoid restarting work after failure; keep running costs low
Business analyst	Reviews of the SRS and enriched SRS	Requirements that are complete for the domain; a way to request specific fixes without rewriting the document
Architect / technical lead	Reviews of the architecture and code changes	Diagrams consistent with the plan; a real diff of what changed, not the model's summary of it
Designer	Reviews of the UI/UX output	A rendered preview of each page; design tokens that change only when approved
Security and QA reviewer	Reviews of the security and test reports	Checkable evidence (scan findings, executed tests) rather than a pass/fail claim
End customers	Use the generated store	Safe handling of accounts, orders and payment data

From these roles we derived seven needs that guided the design. Users need to see intermediate decisions before code exists (N1), to intervene at any stage by approving, rejecting or asking for revision (N2), and to resume work after an interruption without losing earlier approvals (N3). They need security and correctness to be backed by checkable evidence (N4), specifications that cover the conventions of the e-commerce domain (N5), and a trail from each requirement to the artefacts built from it (N6). Finally, because the target users include small teams,

the system should be affordable to run (N7), which is why the default model runs locally.

These needs were derived from the literature in Section 1.1 and refined through the review rounds carried out during the case study.

1.3 Alignment with the Sustainable Development Goals

The project relates most directly to Sustainable Development Goal (SDG) 9, Industry, Innovation and Infrastructure, and in particular to its aim of enhancing research and upgrading the technological capabilities of industrial sectors. AutoForge contributes a research prototype and an evaluated method for using AI in software engineering in a controlled way, built entirely on open tools that a local team can run on its own hardware.

There is also a link to SDG 8, Decent Work and Economic Growth, which includes raising productivity through technological upgrading and supporting the growth of small and medium-sized enterprises (SMEs). Many small businesses want an online store but cannot afford a large development team, and many small development firms build such stores for clients. A pipeline that produces reviewed, tested features with a measurable review cost could help these firms deliver more work without lowering quality.

We are careful not to overstate this. AutoForge has been evaluated on three features in a controlled setting, not deployed with real businesses, so any contribution to these goals is potential rather than demonstrated. Section 7.5 returns to this question, including possible negative effects.

1.4 Purpose and scope of the common report

This common report presents AutoForge as one integrated solution. It explains the shared problem, how the work was divided into four components, how those components were joined into a single pipeline, and how the complete system was evaluated. Detailed research and development evidence for each component, such as the internal design of an individual agent, its prompts and its component-level experiments, is reported in the four individual theses.

The report follows the structure required by the module. Chapter 2 defines the overall problem and the four subproblems. Chapter 3 describes the solution structure and summarises each component. Chapter 4 explains how the components were integrated, and Chapter 5 covers implementation and project execution. Chapter 6 presents the integrated evaluation, Chapter 7 considers commercialization and responsible innovation, and Chapter 8 concludes.

2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS

2.1 Overall problem definition

Current LLM-based tools for building software either generate an application in one step, leaving nothing in between for a person to inspect, or split the work among agents that review each other without giving a human a durable point of control. In both cases, an early mistake in the requirements or the design can travel all the way into the code before anyone sees it, and if the process is interrupted, the work done so far is usually lost.

Stated as a single problem: there is no widely reported way to carry a web application feature through the full software development life cycle with LLM agents such that every stage produces a versioned artefact that a human can approve, reject or send back for revision, the process can pause indefinitely and resume after a restart, the specification is grounded in domain knowledge, and the final code is verified by executable security and quality checks rather than by the model's own judgement.

We have tried to keep this statement separate from our solution. The problem is about control, durability, grounding and verification. AutoForge uses technologies (LangGraph, MongoDB, ChromaDB and others) to address it, but the contribution we claim is the stage-wise, checkpointed, human-approved process itself, not any individual framework, database, retrieval system or model.

2.2 Root causes and overall research gap

Looking across the systems reviewed in Chapter 1, we see four root causes behind the problem.

The first is that "human in the loop" usually means review after the fact. In ChatDev, MetaGPT and the SWE-bench-style coding agents, a person reads the final diff or the output of a completed run. We did not find a system in which every stage output is treated as an independently versioned artefact with its own acceptance, reject or revise decision [2], [3], [5]. This is why problems are found late.

The second is that the process state is not durable. When a long-running multi-agent process is interrupted, whether by a crash, a server restart or simply a reviewer who will not be available until the next day, the run is generally lost rather than paused. A human gate is not practical if the system cannot wait for a human.

The third is that specifications rely on the model's memory. Role-based systems build their specification-equivalent documents from the model's parametric knowledge without retrieving domain knowledge in the sense established by Lewis et al. [14]. Requirements-engineering reviews identify this as under-explored [15]. The result is a specification that may look complete but misses domain conventions.

The fourth is that verification and traceability are weak and disconnected. Most traceability work still covers one narrow link type rather than the chain from requirement to code [16], [17], and most LLM-based testing and security work lacks a deterministic verification layer [18], [19]. Table 2.1 summarises how representative systems compare on the properties that matter here.

Table 2.1: Comparison with related multi-agent software engineering systems

System	Multi-agent	SDLC coverage	Human gates	Artefact trace
ChatDev [2]	Yes	Partial	N/R	N/R
MetaGPT [3]	Yes	Partial	N/R	Partial
SWE agent [5]	Partial	Coding only	N/R	N/R
OpenHands [6]	Yes	Coding	N/R	N/R
AgentCoder [11]	Yes	Coding + testing	N/R	N/R
AutoForge (ours)	**Yes**	**7-agent pipeline**	**Yes**	**Versioned IDs**

Partial indicates that the reviewed work addresses only part of the corresponding SDLC scope. N/R means not explicitly reported in the reviewed publication.

The overall research gap is therefore the combination of these four properties in one working pipeline. Individual pieces exist elsewhere: role agents in ChatDev and MetaGPT, planning-stage oversight in HULA [13], retrieval in RAG systems, and test loops in AgentCoder. What we did not find is a system that treats every SDLC stage as an approvable, versioned artefact, persists its state durably enough to resume after any delay or restart, enriches the specification with retrieved domain knowledge before downstream stages use it, and treats security and testing as stages that produce

genuine, checkable evidence. Closing that gap cannot be done by one agent. It needs the whole chain, which is why the project was divided into four connected components.

2.3 Subproblem and component mapping

We split the overall problem into four subproblems that do not overlap but depend on each other. Each subproblem became one team member's component. The split follows the natural order of the SDLC, so that each component consumes the approved output of the one before it. Table 2.2 shows the mapping.

Table 2.2: Subproblems mapped to components

Component	Subproblem	Agents and parts owned
C1: Santhuka D.N.M.D.	Turning a rough requirement into a complete, domain-grounded specification, and making every stage pause for a human and survive interruption	Requirement Agent, Domain Agent, orchestration layer
C2: Sansala T.G.B.D.	Turning the approved specification into a consistent architecture plan and a reviewable interface design	Architecture Agent, UI/UX Agent
C3: Dissanayake T.C.	Implementing the approved design as real code whose changes can be inspected, built and merged or discarded safely	Coder Agent
C4: Oberathna R.D.T.D.	Producing checkable security and quality evidence for the generated code	Security Agent, QA Agent

All four are needed. Without Component 1 there is no specification and no way to pause and resume. Without Component 2, the Coder Agent would have to invent the structure and the screens while writing code, which is exactly the kind of hidden decision the project is trying to expose. Without Component 3 there is nothing to verify, and without Component 4 the pipeline would end with code whose safety and correctness are only claimed, not shown. The components also depend on each other in practice: Component 4 runs its scans and tests inside the sandbox that Component 3 creates, and every component relies on the gate and checkpoint mechanism built in Component 1.

2.4 Overall aim and group objectives

The overall aim of the project is to design, build and evaluate a multi-agent pipeline that carries an e-commerce feature from a rough requirement to verified code, in which every stage is reviewed by a human, every intermediate artefact is versioned, and the process can recover from interruption.

We state three high-level objectives: a staged pipeline in which every SDLC artefact is versioned and human-reviewed before the next stage begins (O1); durable, resumable stage transitions, so a decision taken well after a stage finishes, even after a backend restart, resumes the pipeline from the exact paused state (O2); and verification of the whole pipeline, security and testing included, on real generated features rather than by describing the design alone (O3). For the group report we broke these into seven measurable objectives, shown in Table 2.3 together with the components responsible and the evidence used to judge them.

Table 2.3: Group objectives, responsible components and evaluation evidence

ID	Group objective	Components	Evidence used
GO 1	Every one of the seven stages produces a versioned artefact and receives an explicit human decision before the next stage starts (O1)	C1-C4	Artefact versions and approval outcome per stage (Table 6.3)
GO 2	An interrupted pipeline resumes at the correct next stage without losing earlier approvals (O2)	C1	Controlled recovery trials (Table 6.9); real backend failure
GO 3	Retrieval-based enrichment produces a more complete specification than generation without retrieval	C1	RAG ON/OFF ablation (Table 6.5)
GO 4	Architecture and interface artefacts pass automatic consistency checks and are visually reviewable before coding	C2	Stage outcomes and revisions (Table 6.3)
GO 5	Every code change is built in a sandbox and merged only after approval	C3	Coder stage outcome (Table 6.3); QA results (Table 6.4)
GO 6	Security and QA produce measurable evidence: detection precision and recall, test pass counts and coverage	C4	Tables 6.6 and 6.7
GO 7	The pipeline works on features of increasing complexity, and the cost of human review is measured (O3)	C1-C4	Tables 6.4, 6.8 and 6.10

2.5 Scope assumptions and constraints

The scope of the complete solution is the development of individual features of an e-commerce web application, from requirement to tested code. We evaluated it on three features: Item Listing, which is a basic create, read, update and delete (CRUD) feature; Authentication and Authorization, which adds identity and access control; and Shopping Cart, which adds stateful business logic. Each feature runs as its own pipeline thread.

The generated applications target one technology stack: a Next.js client using the App Router and TypeScript, an Express-family API, and MongoDB with Mongoose for persistence. Supporting other stacks is outside the current scope. The QA Agent focuses on plain TypeScript modules under the lib/ and models/ folders, because testing interface components or route handlers would need a DOM renderer or a running Next.js process, which the current implementation does not provide. Code that is out of reach is listed in the report as out of scope rather than silently skipped.

Several assumptions underlie the design. We assume that a human reviewer is available for each stage, although not necessarily immediately; the pipeline is built to wait. We assume that the reviewer has enough knowledge to judge the artefact in front of them. We also assume that the local machine can run the default model through Ollama, and that Docker is available for the sandbox.

The main constraints were time, hardware and evaluation scale. As a final-year team we evaluated three features in one domain with one default model, which is enough to show feasibility and to measure the effect of key design choices, but not enough for benchmark-style statistics. There is no shared benchmark yet for going from a requirement to merged, human-approved code, so we could not compare against one.

3 OVERALL SOLUTION STRUCTURE

3.1 Integrated solution overview

AutoForge is a single pipeline, not four separate tools. A user starts a run for one feature, for example "Item Listing" in a sample e-commerce project, and the feature then moves through seven agents in a fixed order: Requirement, Domain, Architecture, UI/UX, Coder, Security and Quality Assurance (QA). Each agent produces one artefact. The pipeline stops after every artefact and waits for a person to approve it, reject it or ask for a revision. Only an approved artefact is passed to the next agent.

The central value of the system is that nothing important happens out of sight. The requirement becomes an SRS that a business analyst can read. The SRS is enriched with retrieved domain knowledge, and the additions are flagged inline so they can be checked. The architecture appears as a plan and Unified Modeling Language (UML) diagrams, the interface appears as rendered page previews, and the code appears as a real git diff. The security and QA stages report concrete findings and executed tests. Each of these is stored as a versioned artefact, so the history of a feature, including every rejected or revised version, can be followed afterwards.

The end-to-end outcome is a feature that has been specified, designed, implemented, scanned and tested, together with a complete record of the human decisions that shaped it. Because the pipeline state is checkpointed after every stage, that outcome does not depend on the process running uninterrupted from start to finish. Table 3.1 lists the key functional requirements of the integrated solution.

Table 3.1: Key functional requirements of the integrated solution

ID	Functional requirement	Component
FR-01	Generate an SRS from a rough requirement using a schema-constrained LLM call and check it against a gap-analysis checklist	C1
FR-02	Enrich the approved SRS with domain knowledge retrieved from a knowledge base, flagging every addition inline	C1
FR-03	Produce an architecture plan and UML use-case, sequence and class diagrams, and check plan coverage and diagram consistency	C2

ID	Functional requirement	Component
FR-04	Generate page-level UI metadata and components, and render a preview screenshot of each page	C2
FR-05	Implement the approved design on an isolated git branch and build it in a sandbox before review	C3
FR-06	Present the code change as a diff, manifest and merge report built from the real git diff	C3
FR-07	Scan generated code with pattern, secret and dependency scanners and apply a severity gate	C4
FR-08	Generate and execute tests for testable modules and report pass/fail counts and out-of-scope files	C4
FR-09	Pause after every stage and accept an approve, reject or request-revision decision	C1
FR-10	Store every stage output as a versioned artefact with Markdown and, where relevant, PNG or JSON versions	C1-C4

3.2 High-level architecture

Figure 3.1 is the system diagram. It shows the Requirement Agent working with a business analyst to produce the SRS, and the six agents that follow it, each with its own output and a human approval mark. The dashed lines are feedback loops: at any stage, a reviewer can send work back rather than let it move forward. Every agent's output is also collected into the final project outputs: the specification, the architecture, the UI/UX design, the source code, the security report and the test report.

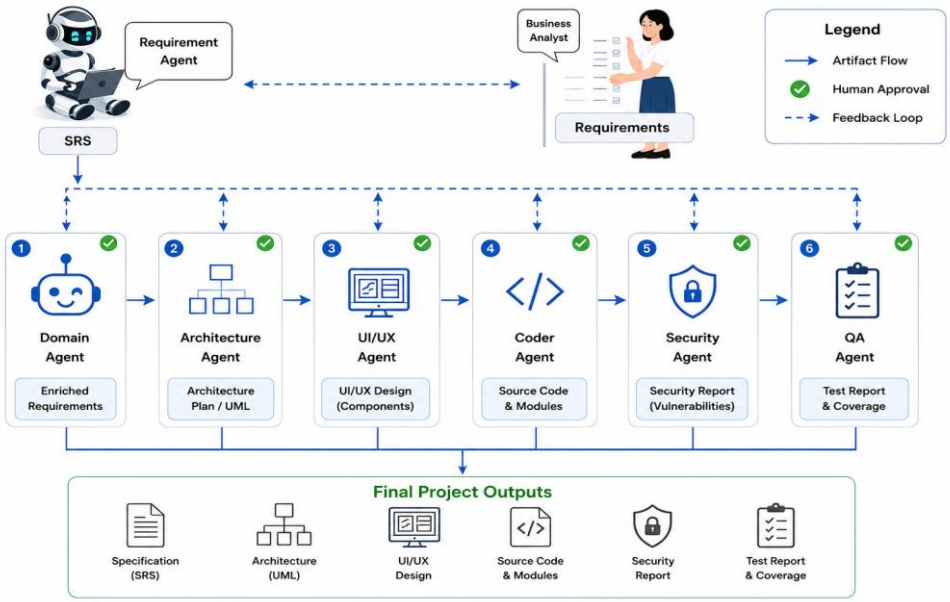


Figure 3.1: AutoForge system diagram

3.3 Component 1: Requirement Generation & Domain Enrichment

Component 1 is owned by Santhuka D.N.M.D. Its subproblem is to turn a rough requirement into a complete, domain-grounded specification, and to provide the pause-and-resume mechanism that every other stage uses.

The input is the user's initial requirement, refined with a business analyst. The Requirement Agent generates the first SRS from one schema-constrained LLM call, so the model must fill a fixed structure instead of writing free text. The structured result is converted to Markdown, and a second, narrower LLM call checks it against a gap-analysis checklist before a person sees it. When the reviewer asks for a change, the agent patches the specific fields involved instead of regenerating the whole document.

The Domain Agent then embeds the approved SRS, retrieves the closest matching chunks from a ChromaDB-indexed knowledge base, and asks the LLM for a small, citation-grounded enrichment plan. A deterministic step, with no model involved, merges that plan onto a copy of the SRS and produces an Enhanced SRS in which every addition is flagged inline. We arrived at this "small plan, deterministic merge" pattern after early testing showed that asking the model to regenerate the full document could silently drop required sections.

The component's third part is the orchestration layer: the LangGraph state graph, the interrupt that pauses it at each stage, and the MongoDB checkpointer. Its outputs are the SRS, the Enhanced SRS and a durable, resumable pipeline. Its contribution to the whole solution is the foundation: every downstream agent reads the approved specification, and every stage depends on the gate and checkpoint mechanism.

3.4 Component 2: Architecture Planning and UI/UX Design Generation

Component 2 is owned by Sansala T.G.B.D. Its subproblem is to turn the approved specification into a consistent technical design and a reviewable interface before any code is written.

Its input is the approved Enhanced SRS. The Architecture Agent produces an architecture plan from it, derives UML use-case, sequence and class diagrams,

checks that the plan covers the specification and that the diagrams are consistent with each other, and renders the diagrams to PNG images through PlantUML.

The UI/UX Agent loads the specification, the architecture plan and a shared design system. It generates and checks page-level UI metadata, then generates each component behind a content-quality gate with a limited repair loop, so a failing component is regenerated without discarding the rest. It assembles the pages and renders a preview screenshot of each one. New design tokens are written only once the design is approved, so a rejected run cannot affect a later feature.

The outputs are the architecture plan with its diagrams, and the wireframes, UI metadata and components. The contribution to the whole solution is that the Coder Agent receives a fixed plan and a fixed interface to implement, instead of having to invent them while it writes code.

3.5 Component 3: Code Generation with Sandboxed Verification

Component 3 is owned by Dissanayake T.C. Its subproblem is to implement the approved design as real code, in a way that keeps the change inspectable and keeps the main codebase safe until a person agrees.

Its inputs are all of the approved artefacts so far. The Coder Agent plans a change against them, re-prompting itself when a validator finds that the plan does not cover something it should. It then opens an isolated feature branch and runs an agentic tool-calling loop to implement the plan, in which the model reads and writes files and runs commands rather than just describing code. It commits the change, runs a sandboxed install, build, lint and test pass, and builds the diff, the manifest and the merge report from the real git diff, not from the coding loop's own account of what it did. On approval the branch is merged; on rejection it is discarded.

The generated application targets a Next.js client (App Router, TypeScript) with an Express-family API and MongoDB/Mongoose persistence. The output is a code manifest and merge report for a change that has already been built in a container. The contribution to the whole solution is that the code a reviewer approves is exactly the code that exists, and it has already been shown to build.

3.6 Component 4: Security Analysis and Quality Assurance

Component 4 is owned by Oberathna R.D.T.D. Its subproblem is to produce security and quality evidence for the generated code that a reviewer can check for themselves.

Its input is the code change built by Component 3. The Security Agent runs three deterministic scan layers. A pattern scanner parses each file's Abstract Syntax Tree (AST) using the TypeScript compiler and flags eval, dynamic code execution, unsafe HTML injection and unsafe bracket-style access. A secret scanner looks for exposed credentials, and a dependency scanner runs npm audit against the sandbox's lockfile. Findings are deduplicated into a pass/fail gate in which a critical or high-severity finding fails the gate. An optional LLM layer gives the findings a second read, following the advice of Zhu et al. [19] to combine static analysis with LLM review; if the model cannot be reached, this layer is marked as skipped rather than failing the scan.

The QA Agent looks for testable modules, concentrating on plain TypeScript files under lib/ and models/. It detects the test framework from package.json and defaults to Node's built-in test runner. Test generation tries the LLM first and falls back to a deterministic template for two recognized code shapes, and the agent records which method produced each test file. The tests run in the same sandbox the Security Agent uses. The outputs are a security report and a QA report, both of which pause for a human decision. The contribution to the whole solution is that the pipeline ends with evidence, not with a claim that the code is safe and correct.

Table 3.2: Summary of the four components

	C1	**C2**	**C3**	**C4**
Input	Rough requirement	Approved Enhanced SRS	All approved design artefacts	Built code change
Core method	Schema-constrained generation; retrieval with deterministic merge; checkpointed state graph	Plan and UML with consistency checks; UI generation with repair loop and previews	Agentic tool-calling on an isolated branch; sandboxed build	AST, secret and dependency scans with severity gate; generated and executed tests

	C1	C2	C3	C4
Output	SRS, Enhanced SRS, resumable pipeline	Plan + UML; wireframes, metadata, components	Code manifest, diff, merge report	Security report, QA report

3.7 End to end user and data flow

A typical run for the Item Listing feature shows how the components work together. A user creates a run and describes the feature. Component 1's Requirement Agent produces SRS version 1, and the graph pauses. In our case study, the reviewer asked for two additional user stories for Admin and Manager roles, so the agent patched the SRS to version 2, which was approved. The approval resumes the graph, and the Domain Agent produces the Enhanced SRS, which was approved as it stood.

The Enhanced SRS then crosses into Component 2. The Architecture Agent's plan needed one correction before approval: a technical component had been placed in a UML diagram as an actor, and the reviewer sent it back. The UI/UX Agent's output went through component-level revisions before approval. At this point the Coder Agent in Component 3 has a fixed, approved specification, architecture and interface to work from.

The Coder Agent implements the change on a feature branch. During review, the reviewer noticed that the generated item model required a manually specified identifier, even though MongoDB generates identifiers automatically. The revision removed it and updated the model, the API and the frontend together, and the fourth version was approved and merged. The merged code then passes to Component 4: the Security Agent scanned it and found no issues, and the QA Agent generated and ran two tests, both of which passed. Each of those reports was also approved by a person.

Throughout this flow, the data that moves between components is always an approved, versioned artefact, and the state of the run, including which stage it is at and which versions were approved, is saved in MongoDB after every step. If the backend stops at any point, the run continues from the last checkpoint once the system is restarted. Chapter 4 describes these interfaces and the failure paths in more detail.

4 COMPONENT INTEGRATION AND COHESIVE OPERATION

4.1 Integration strategy

We integrated AutoForge around artefacts rather than around direct calls between agents. No agent calls another agent. Each agent reads the approved artefacts it needs, produces its own artefact, and hands control back to the orchestration layer, which records the result, saves a checkpoint and waits for a human decision. This kept the four components loosely coupled: a team member could change the internals of an agent without breaking the others, as long as the artefact it produced kept the agreed shape.

Integration broadly followed the pipeline order, because every stage depends on the gate and checkpoint mechanism and on the approved output of the stage before it. Each agent was first made to produce its artefact on its own and was then connected as a node in the LangGraph state graph. LangGraph was chosen for this layer because it treats a multi-agent workflow as an explicit state graph with built-in support for pausing and checkpointing [20], which is exactly what a human gate needs.

Ownership of the interfaces followed the component split. Component 1 owned the orchestration layer and therefore the shape of the gate decision and the run state. Each component owned the format of the artefacts its agents produce. The one deliberate point of shared infrastructure was the sandbox: rather than have the Security and QA agents maintain their own way of running commands, Component 3's sandbox is passed to them as a dependency-injected parameter. Both npm audit and the test suite run inside the same container the code was built in, so the evidence they produce is about exactly the code that was built.

Several integration risks appeared during development, and most of the design decisions described in Chapter 3 are answers to them. An earlier model struggled to follow the pipeline's structured JSON schemas reliably, which broke the hand-over between agents; we switched the default to qwen3-coder. Full regeneration of the specification sometimes dropped required sections without warning; the Domain Agent moved to the small-plan, deterministic-merge pattern. A model's description

of its own code changes could not be trusted as a record; the Coder Agent's report is built from the real git diff. And long-running stages combined with human pauses made interruptions likely; every transition is checkpointed.

4.2 Interface data contracts and shared models

The main data contract in AutoForge is the versioned artefact. Every agent's output is saved with a version number and an addressable identifier, as Markdown for the reviewer and, where it applies, as JSON or PNG alongside it. The Markdown form is what a person reads; the JSON form is what the next agent consumes; the PNG form holds diagrams and page previews. Keeping these twins together means the reviewer and the next agent are always looking at the same version. Table 4.1 lists the artefacts that flow between components.

Table 4.1: Artefact contracts between stages

Producer	Artefact	Representations	Consumed by
Requirement Agent (C1)	SRS	Markdown + structured JSON	Domain Agent and all later stages
Domain Agent (C1)	Enhanced SRS with inline flags	Markdown + JSON	Architecture, UI/UX, Coder
Architecture Agent (C2)	Architecture plan + UML	Markdown + PNG diagrams	UI/UX, Coder
UI/UX Agent (C2)	Wireframes, UI metadata, components	Metadata (JSON), components, PNG previews	Coder
Coder Agent (C3)	Code manifest, diff, merge report	Markdown + git branch	Security, QA
Security Agent (C4)	Security report with gate decision	Markdown + findings	Reviewer; Coder on revision
QA Agent (C4)	QA report	Markdown + pass/fail counts	Reviewer; Coder on revision

The second contract is the gate decision. At every stage the reviewer has the same three choices: approve, reject or request a revision. An approval resumes the graph at the next stage. A revision request sends the stage back to its agent, which produces a new version; the older versions are kept. A rejection discards the output, and in the Coder stage this means the feature branch is dropped without touching the main

codebase. Because the same decision contract applies to all seven agents, the review interface does not need special logic for each stage.

The third contract is the run state held by the orchestration layer. For each feature it records which stage the pipeline is at, the versions produced so far and the decisions taken on them. LangGraph saves this state to MongoDB at every transition, so the approval history and the pipeline position are stored together in one checkpoint. This matters for recovery: after a restart, the system cannot "forget" an approval while still remembering the stage, or the other way round.

AutoForge also depends on several external interfaces. Table 4.2 lists them with their purpose and how failures are handled.

Table 4.2: External interfaces and their failure handling

Interface	Used by	Purpose	Failure handling
Ollama (local) / OpenAI API	All LLM-using agents	Text and structured generation through one provider abstraction	Schema validation; re-prompting or repair loops; provider can be switched
ChromaDB	Domain Agent	Store and retrieve domain knowledge chunks	Enrichment is a small, reviewable plan; reviewer sees every addition
MongoDB	Orchestration layer	Checkpoint run state and decisions	State survives process restarts; resume from last checkpoint
PlantUML	Architecture Agent	Render UML diagrams to PNG	Diagrams checked for consistency before review
Playwright + headless Chromium	UI/UX Agent	Render page preview screenshots	Failing component regenerated by the repair loop
Git	Coder Agent	Isolate each change on a feature branch	Branch discarded on rejection
Docker sandbox	Coder, Security, QA	Install, build, lint, audit and test	Build result shown to the reviewer before merge
TypeScript compiler, npm audit	Security Agent	AST pattern scan; dependency audit	Critical or high findings fail the gate
Node.js test runner	QA Agent	Execute generated tests	Untestable files listed as out of scope

4.3 Shared nonfunctional requirements

Some qualities cannot be owned by one component; they only hold if all four components support them. Table 4.3 lists the shared non-functional requirements (NFRs), how the integrated system addresses each one, and the evidence for it. Where a quality is not yet backed by measurement, the table says so.

Table 4.3: Shared non-functional requirements and evidence

NFR	How AutoForge addresses it	Evidence
Reliability and recoverability	State checkpointed to MongoDB at every transition; resume from the exact paused node	7/7 controlled recovery trials; one real backend failure recovered (Section 6.3.7)
Security	AST, secret and dependency scanners with a severity gate; optional LLM second read	87.5% precision and 77.8% recall on injected vulnerabilities (Section 6.3.4)
Testability and quality	Generated unit, regression and integration tests executed in the sandbox	79% overall coverage, 54 of 60 tests passed (Section 6.3.5)
Traceability	Every artefact versioned and addressable; every decision recorded	Version and approval history for all stages (Table 6.3)
Performance and cost	Time, LLM calls, retries and tokens logged per run; local default model	31-57 minutes and 25,400-47,900 tokens per run (Section 6.3.8)
Usability for reviewers	Readable Markdown, rendered diagrams and page previews, real diffs	Review effort measured (36 minutes); no formal usability study yet
Maintainability	Each agent an isolated Python package with its own prompt, schema and validators; shared provider abstraction	Deterministic parts tested against hand-built fixtures
Portability	All build, audit and test steps run inside a Docker container	Same sandbox reused by three agents
Privacy	Default model runs locally, so project data need not leave the machine	Design property; not separately measured

Two of these deserve a comment. Performance is the quality we are least able to claim. The gated pipeline is measurably slower and uses more model calls than the baseline run, and part of that cost is the reviewer's time. We treat this as a known trade-off rather than a defect, and Chapter 6 measures it. Usability, on the other hand, has not yet been measured with independent users, so the evidence column only records review effort, not satisfaction or ease of use.

4.4 Integrated workflow and exception handling

Figure 4.1 shows the cycle that every stage follows. The agent takes the approved inputs from earlier stages and generates its artefact. Automatic checks run next; depending on the stage these are schema validation, plan coverage and diagram consistency, the sandboxed build, or the security gate. The artefact is saved as a new version and the state is checkpointed. The graph then pauses through LangGraph's interrupt and waits for a human, which may take minutes or days. The reviewer's decision moves the pipeline forward, sends the stage back for a new version, or stops it.

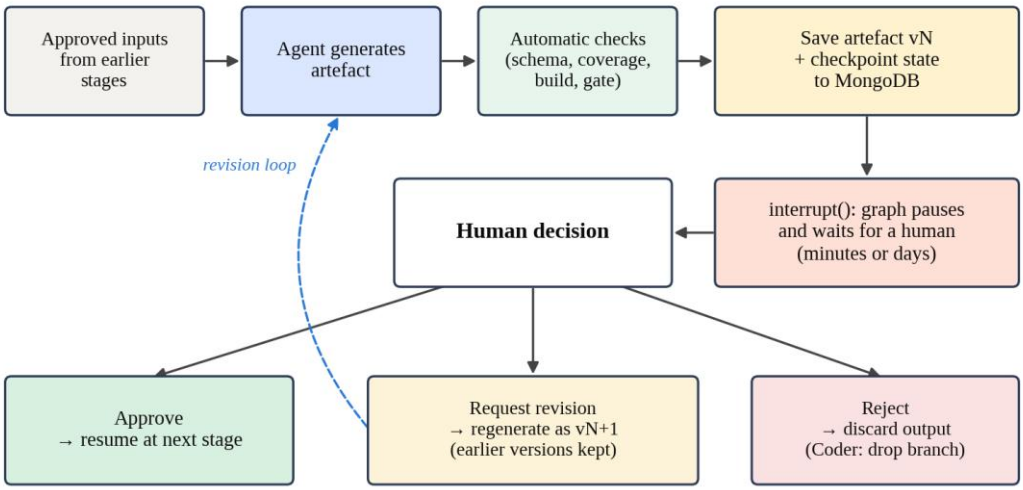


Figure 4.1: Integrated workflow at every stage

The normal path is the one described in Section 3.7. The more interesting part of the design is what happens when something goes wrong. Table 4.4 lists the failure paths we designed for and how the system responds to each.

Table 4.4: Failure paths and recovery behaviour

Situation	System behaviour
Backend stops or restarts mid-run	Run resumes from the last MongoDB checkpoint at the exact paused node, with earlier approvals intact
Reviewer is unavailable for a long time	Graph stays paused; no work is lost while it waits
Reviewer requests a revision	Same agent produces version N+1; earlier versions remain available for comparison
Reviewer rejects the Coder stage	Feature branch is discarded; main codebase is unchanged
LLM output does not match the schema or misses part of the plan	Requirement output is schema-constrained; Coder re-prompts itself when its validator finds a coverage gap
A generated UI component fails its quality gate	Only that component is regenerated in a limited repair loop; the rest are kept
Generated code does not build	Failure is visible in the sandbox build result before the reviewer approves anything
Critical or high-severity security finding	Security gate fails before the report is produced; reviewer can send the work back to the Coder stage
Optional LLM security reviewer unreachable	LLM layer is marked as skipped; deterministic scan results still stand
Code the QA Agent cannot exercise	Listed in the report as out of scope rather than silently skipped

Two of these paths were exercised directly in the evaluation. During the work, the backend failed unexpectedly in the middle of a run, and the graph resumed from its last checkpoint instead of starting again. Because a single incident does not establish a recovery rate, we then introduced controlled interruptions at all seven stage boundaries; Section 6.3.7 reports the results. The revision path was exercised at four of the seven stages of the Item Listing case study.

4.5 Team contribution and dependency matrix

Table 4.5 maps each team member to their component, their main development tasks and their share of the integration work. The dependency column shows which other components each one relies on. Every member also took part in the shared tasks: integration testing, the evaluation runs reported in Chapter 6. A signed contribution declaration is given in Appendix B.

Table 4.5: Team contribution and dependency matrix

Member	Component and main tasks	Integration work	Depends on
Santhuka D.N.M.D. (IT22098078)	C1: Requirement Agent, Domain Agent, knowledge base, orchestration layer	LangGraph state graph, gate decision, MongoDB checkpointing, artefact versioning	None upstream; all components use C1's gates
Sansala T.G.B.D. (IT22174826)	C2: Architecture Agent, UI/UX Agent, design system, diagram rendering	Architecture and UI artefact formats consumed by the Coder Agent	C1 (approved Enhanced SRS)
Dissanayake T.C. (IT22157232)	C3: Coder Agent, branch management, sandboxed build, review interface pages	Docker sandbox shared with C4; merge on approval	C1, C2
Oberathna R.D.T.D. (IT22102478)	C4: Security Agent, QA Agent, severity gate, test generation and execution	Use of C3's sandbox; reports that can route work back to C3	C3 (built code), C1

5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION

5.1 Common technology stack and infrastructure

The four components share one backend, one orchestration layer, one model provider abstraction, one sandbox and one artefact store. Choosing these together, rather than letting each component pick its own, was what made a single pipeline possible. Table 5.1 lists the main technologies and the reason each was chosen.

Table 5.1: Common technology stack and justification

Concern	Technology	Why it was chosen
Backend	FastAPI (Python)	Keeps the backend in the same language as the agent packages and the orchestration library; well suited to long-running requests such as LLM calls
Orchestration	LangGraph	Models the workflow as an explicit state graph with native interrupt and checkpoint support [20], which is what a human gate needs
Checkpoint store	MongoDB	Lets pipeline state survive a process restart, so a paused run can be resumed days later
Retrieval	ChromaDB [21]	Embedding database for the domain knowledge base, following the retrieve-then-generate pattern of RAG [14]
LLM provider	Ollama with qwen3-coder (default); OpenAI API (alternative)	Local model keeps project data on the machine and avoids per-token fees; qwen3-coder followed the structured JSON schemas reliably after an earlier model did not
Diagrams	PlantUML	Turns text diagram definitions into PNG images, which is easier for a model to produce correctly than drawing
UI previews	Playwright with headless Chromium	Renders real screenshots of generated pages so the reviewer approves what the page looks like, not a description of it
Isolation	Git feature branches	Keeps every generated change away from the main code until it is approved; provides the real diff for review
Sandbox	Docker	Runs install, build, lint, audit and test in a repeatable, isolated environment shared by three agents

Concern	Technology	Why it was chosen
Security scanning	TypeScript compiler (AST), secret patterns, npm audit	Deterministic checks that give the same result every time, as recommended in [19]
Testing	Node.js built-in test runner (default)	Works without extra dependencies; the QA Agent still detects and respects the project's own framework
Generated app	Next.js (App Router, TypeScript), Express-family API, MongoDB/Mongoose	A realistic, widely used web stack; TypeScript types also give the AST scanner and the tests more to work with

The choice of a local model as the default deserves some explanation, because it shaped the rest of the stack. Running qwen3-coder through Ollama meant that every agent could be tested many times during development without paying for API calls, and that a small team's project data would not have to leave their machine. The cost is speed: local inference is slower than a large, hosted model, which is one reason the full pipeline takes between half an hour and an hour per feature (Section 6.3.8). Because all LLM calls go through one provider abstraction, a team that prefers a hosted model can switch to the OpenAI provider without changing any agent code.

5.2 Development and integration process

Each agent was developed as its own isolated Python package containing an entry point, a prompt, a schema and the stage-specific modules it needs, such as validators, Markdown builders, scanners or tool factories. This structure let the four members work on their components in parallel in a shared Git repository, with each person's work concentrated in their own agent packages, while the orchestration layer tied the packages together.

We tested the two kinds of code in the system differently. Deterministic parts, including validators, the diff and Markdown builders and the scanner rule matching, were tested against hand-built fixtures with no model call involved, so these tests are fast and give the same answer every time. Parts that call an LLM cannot be tested that way, because a response can be perfectly valid against its schema and still be

wrong in substance. Those pipelines were therefore checked by hand against real feature data at each milestone.

Within the system itself, the same discipline applies to generated code. Every change the Coder Agent makes is isolated on a git branch, built in the sandbox and merged only on approval. In effect, AutoForge applies to its own output the review-before-merge practice that we followed when developing it.

5.3 Deployment and operational configuration

The current deployment runs on a single machine. The FastAPI backend hosts the orchestration layer and the agent packages. Ollama serves the default model locally, MongoDB stores the checkpoints, ChromaDB stores the domain knowledge base, and Docker provides the sandbox in which generated applications are built and tested. The choice of provider and model is a configuration setting rather than a code change, because of the provider abstraction.

Operationally, two mechanisms take the place of a conventional backup and restore process. The MongoDB checkpoints hold the state of every run, so restarting the backend restores each run to its last paused point. The versioned artefact store holds every version of every artefact, including rejected and revised ones, so earlier work can be inspected or compared at any time. For monitoring, each run records its total execution time, number of LLM calls, retries and token usage from the execution logs, which is where the performance figures in Chapter 6 come from.

A multi-user, server-based deployment is not part of the current work. Moving to one would need authentication for reviewers, a shared model server or hosted model, and proper backup of the MongoDB and ChromaDB stores. We discuss this further in Chapters 7 and 8.

5.4 Evidence of team execution

The shared GitHub repository provides evidence of the team's development activities and individual contributions. Its history shows each member's commits concentrated in their assigned components, including the requirements, domain and orchestration code for Component 1, the architecture and UI/UX agents for Component 2, the

coder agent and review interface for Component 3, and the security and QA agents for Component 4. The history also demonstrates collaborative work across component boundaries during integration, particularly on the shared sandbox and review interface. The repository can be accessed at **[GitHub Repository Link].**

Project tasks and progress were tracked using Microsoft Planner, while Microsoft Teams was used for communication and coordination. The team conducted weekly meetings to discuss progress, technical issues, and upcoming tasks. In addition, meetings were held with the Supervisor once per month and the Co-supervisor twice per month to review progress and receive feedback. Further supporting evidence from project management records, meeting records, and individual research diaries is provided in Appendix C.

6 INTEGRATED TESTING EVALUATION AND VALIDATION

6.1 Evaluation framework and success criteria

AutoForge is a system-building project, so we followed the design-science paradigm described by Hevner et al. [22]: build an artefact, then evaluate it in the environment it is meant for. We did not use a benchmark-based evaluation because no shared benchmark yet exists for the task of going from a requirement to merged, human-approved code. Instead, the evaluation is a controlled case study in a real sample e-commerce project, using features of different complexity.

The evaluation had three parts. First, one feature, Item Listing, was carried through all seven human-gated stages. Second, at each stage we recorded the artefact produced, what the reviewer saw and the decision that resumed the graph, including any requested revision. Third, the code, the security scan and the tests were verified mechanically through a sandboxed build and real execution, not by inspection alone. Further experiments then extended the evaluation to two more complex features and isolated the effect of three design choices: retrieval, human gates and checkpointing.

The evaluation is organized around five research questions (RQs). Table 6.1 lists them, the metrics used for each, and the success criterion we applied.

Table 6.1: Research questions, metrics and success criteria

RQ	Question	Metrics	Success criterion
RQ 1	Can the pipeline carry a feature through all seven human-approved stages while producing the required artefacts?	Artefact completeness; gate correctness; build pass/fail	All seven stages produce a versioned artefact with a recorded human decision
RQ 2	How does stage-wise human approval affect quality and revisions?	Revisions; QA pass rate; security issues; escaped defects; review effort	Measurable quality difference between gates ON and OFF, with the cost reported
RQ 3	Does the pipeline reliably resume from a persisted checkpoint?	Correct resume per interrupted stage; recovery rate	Resume at the expected next stage with earlier approvals kept, in every trial

RQ	Question	Metrics	Success criterion
RQ 4	Do Security and QA produce genuine, checkable evidence?	TP, FP, FN, precision, recall; tests executed, passed, failed; coverage	Real scan findings and executed tests, measured against a controlled set
RQ 5	Does domain retrieval run as a reviewable enrichment stage, and does it help?	Requirements generated; useful and missing requirements; revisions	Retrieval ON improves completeness over retrieval OFF

Alongside these quality measures we recorded the cost of running the pipeline: total execution time, number of LLM calls, retries and token usage for each run. A pipeline with more stages and more pauses will always cost more than a single generation step, so the question is not whether it costs more but whether the quality gained is worth the extra cost.

6.2 Integration and end to end testing

Testing the integrated system meant checking that the components behave correctly together, not only that each agent works on its own. Table 6.2 lists the integrated test scenarios, what each one checks and where the evidence for it is reported. Some scenarios are covered by measured results in this chapter; others describe designed behavior that was exercised during development but not measured separately, and the table says which.

Table 6.2: Integrated test scenarios

ID	Scenario	Expected result	Evidence
IT-01	Run one feature through all seven stages	Seven versioned artefacts, each with a human decision	Table 6.3
IT-02	Reviewer requests a revision	New version produced; earlier version kept; graph resumes on approval	Four stages revised (Table 6.3)
IT-03	Approval resumes the paused graph at the next stage	Next agent starts only after approval	All stages (Table 6.3)
IT-04	Backend interrupted after a stage's state is saved	Resume at the expected next stage with prior approval kept	7/7 trials (Table 6.9)
IT-05	Unplanned backend failure mid-run	Resume from last checkpoint, no restart	Observed once (Section 6.3.7)

ID	Scenario	Expected result	Evidence
IT-06	Coder change built in the sandbox before review	Install, build, lint and test run before the diff is shown	Coder stage (Table 6.3)
IT-07	Security scan on code with injected vulnerabilities	Known vulnerabilities detected and classified	Table 6.6
IT-08	Tests generated and executed by category	Pass/fail counts and coverage per category	Table 6.7
IT-09	Pipeline on features of increasing complexity	All stages complete for each feature	Table 6.4
IT-10	Same requirement with retrieval ON and OFF	Difference in completeness measured	Table 6.5
IT-11	Pipeline with human gates ON and OFF	Difference in quality and cost measured	Table 6.8
IT-12	Module the QA Agent cannot test	Listed as out of scope in the QA report	Designed behaviour; not measured

6.3 Overall results

This section presents the group-level results. Component-level experiments, such as prompt variations within a single agent, are reported in individual theses.

6.3.1 Item Listing (CRUD) case study

The first evaluation traced the Item Listing feature through all seven stages. Table 6.3 shows the final version of each artefact and its approval outcome.

Table 6.3: Item Listing (CRUD): final artefact version &approval

Stage	Artefact	Version	Approval	Status
Requirement	SRS	v2	Human	Revised and approved
Domain	Enhanced SRS	v1	Human	Approved
Architecture	Plan + UML	v3	Human	Revised and approved
UI/UX	Wireframes + metadata	v3	Human	Revised and approved
Coder	Code manifest	v4	Human	Revised and approved
Security	Security report	v2	Human	Pass (0 findings)
QA	QA report	v1	Human	Pass (2/2 tests)

All seven stages executed and produced a reviewable artefact, which answers RQ1 for this feature. Four of the seven stages needed a human-requested revision before

they were approved, and each revision fixed a concrete problem. At the Requirement stage, the reviewer added two user stories for Admin and Manager roles that the first SRS had missed. At the Architecture stage, a technical component had been drawn as an actor in a UML diagram and had to be corrected. The UI/UX stage needed component-level fixes. At the Coder stage, the generated model required a manually specified identifier even though MongoDB generates one automatically; removing it meant updating the model, the API and the frontend together. The Domain, Security and QA stages were approved without a revision.

These revisions are the first piece of evidence that human approval was not a formality. Each of the four problems would have been carried into every later stage had nobody stopped to look. The identifier problem is a good example: had it reached production, it would have affected the database model, the API and the screens at once.

6.3.2 Multi-feature evaluation

A CRUD feature is the easiest case for any code generator, so we selected two more features of increasing complexity to test whether the pipeline scales beyond it. Authentication and Authorization adds identity and access control, and Shopping Cart adds stateful business logic. Table 6.4 summarises the results.

Table 6.4: Multi-feature complexity evaluation

Feature	Complexity	Stages revised	QA tests passed	Security result
Item Listing	Low	4	2/2	0 findings
Authentication / Authorization	Medium	2	14/16	1 finding
Shopping Cart	Medium / High	4	18/20	1 finding

The pipeline completed all seven stages for all three features. As complexity rose, the number of tests rose from 2 to 16 and 20, and a small number of tests failed (two in each of the more complex features), which the QA reports made visible to the reviewer rather than hiding. Each of the more complex features had one security finding. Authentication needed revisions at fewer stages than the other two, which

suggests that the number of revisions depends on the particular feature rather than rising steadily with complexity; three features are too few to say more.

6.3.3 Domain retrieval (RAG) ablation

To check whether the Domain Agent's retrieval step actually helps, rather than assuming it does because retrieved text appears in the output, we ran a controlled ablation. The same requirement request was executed with the same model configuration twice, once with retrieval enabled and once with it disabled. Table 6.5 and Figure 6.1 show the results.

Table 6.5: Domain retrieval (RAG) ablation

Metric	RAG ON	RAG OFF
Requirements generated	18	15
Useful domain requirements	14	8
Missing requirements	2	7
Human revisions	3	6

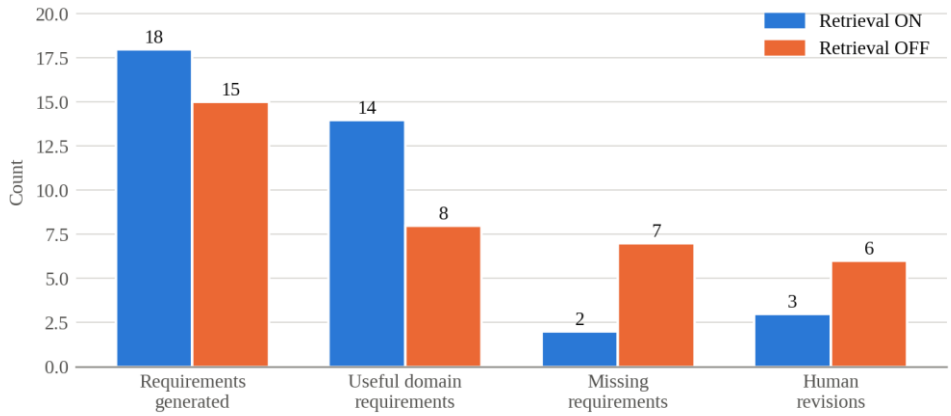


Figure 6.1: Specification completeness with retrieval enabled and disabled

With retrieval enabled, the specification contained more useful domain requirements (14 against 8), missed far fewer requirements (2 against 7), and needed half as many human revisions (3 against 6). The total number of requirements rose only from 15 to 18, so the improvement is not simply a longer document; it is a more relevant one. This answers RQ5: retrieval runs as its own reviewable stage, and it measurably improves completeness for this request.

6.3.4 Security detection

A scan that finds nothing in a clean application says little about whether the scanner works. To measure detection properly, known vulnerabilities were injected into isolated versions of the generated application and the Security Agent's findings were compared against them. Each finding was classified as a true positive (TP, correctly identified), a false positive (FP, reported but not real) or a false negative (FN, missed). Table 6.6 shows the results.

Table 6.6: Security detection metrics

Metric	Result
True positives	7
False positives	1
False negatives	2
Precision	87.5%
Recall	77.8%

Precision of 87.5% means that seven of the eight reported findings were real, so a reviewer is rarely chasing a false alarm. Recall of 77.8% means that seven of the nine injected vulnerabilities were found and two were missed. For a scanner that runs automatically before a human decision, this is useful but not sufficient on its own, which is one reason the security report still goes to a person.

6.3.5 QA coverage

The first Item Listing run produced two passing tests, which is far from complete validation. The expanded evaluation classified tests into unit, regression and integration categories and reported executed, passed and failed counts alongside coverage, as shown in Table 6.7.

Table 6.7: QA coverage by test category

Category	Executed	Passed	Failed	Coverage
Unit	30	28	2	84%
Regression	18	16	2	78%
Integration	12	10	2	74%
Total	**60**	**54**	**6**	**79%**

Of 60 executed tests, 54 passed, with an overall coverage of 79%. Coverage was highest for unit tests and lowest for integration tests, which is what we would expect

given that the QA Agent concentrates on plain TypeScript modules and cannot yet exercise route handlers or interface components. Together with the security results, this answers RQ4: both agents produce real, checkable evidence, measured against known outcomes rather than reported as a status flag.

6.3.6 Human-gates ablation

The revisions in Section 6.3.1 show that reviewers caught real problems, but on their own they cannot show that human approval produces better results than a fully automatic pipeline. To test that, the pipeline was run under two conditions: with human gates on, where reviewers could approve, reject or request revisions, and with them off, where every stage advanced automatically. Table 6.8 and Figure 6.2 compare the two.

Table 6.8: Human gates ON versus OFF

Metric	Gates ON	Gates OFF
Successful pipelines	3/3	2/3
Average revisions	5.0	1.0
QA pass rate	91%	78%
Security issues	1	4
Escaped defects	1	5
Human review effort	36 min	0

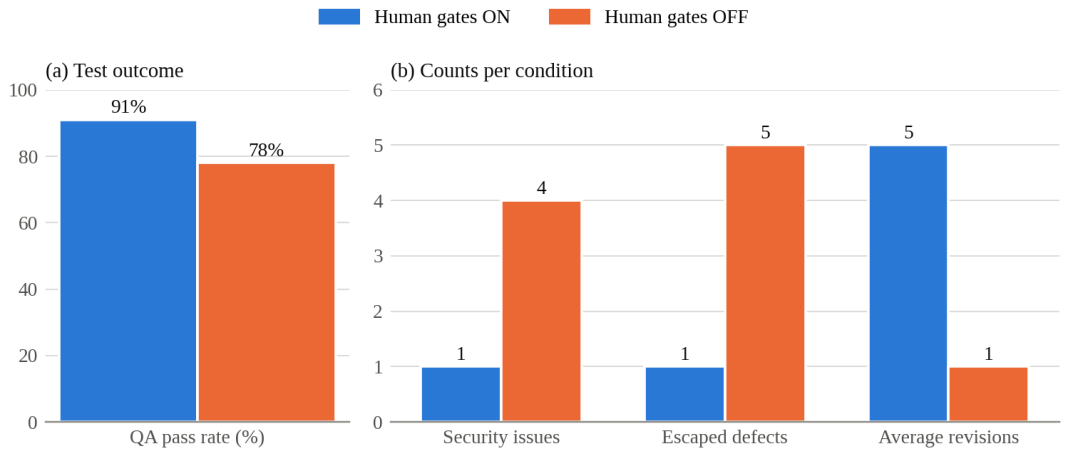


Figure 6.2: Quality outcomes with human gates switched on & off

With the gates on, all three pipelines succeeded against two of three without them. The QA pass rate rose from 78% to 91%, security issues fell from four to one, and

escaped defects fell from five to one. The price was more revision cycles (5.0 on average against 1.0) and 36 minutes of human review effort that the automatic condition did not need. This is the clearest answer to RQ2: human approval cost time, but in return it stopped most of the defects that would otherwise have reached the end of the pipeline.

6.3.7 Checkpoint recovery

During development the backend failed unexpectedly in the middle of a run, and the graph resumed from its last checkpoint instead of starting again. That showed the mechanism could work, but one incident cannot establish a recovery rate. We therefore introduced controlled interruptions at each of the seven stages, after the state had been saved, and counted a recovery as successful only if the pipeline resumed at the correct next stage without losing the earlier approval decision. Table 6.9 shows the trials.

Table 6.9: Controlled checkpoint recovery trials

Trial	Interrupted stage	Expected resume	Actual resume	Correct
1	Requirement	Domain	Domain	Yes
2	Domain	Architecture	Architecture	Yes
3	Architecture	UI/UX	UI/UX	Yes
4	UI/UX	Coder	Coder	Yes
5	Coder	Security	Security	Yes
6	Security	QA	QA	Yes
7	QA	Completion	Completion	Yes

All seven trials resumed correctly, a recovery rate of 7/7, or 100%. Together with the real failure, this answers RQ3 for the conditions tested. Seven trials, one per stage boundary, are enough to show that every transition is protected, but not enough to estimate how the mechanism would behave under every kind of failure; we return to this in Section 6.6.

6.3.8 Execution performance

A multi-agent pipeline adds model calls, artefact processing, approval pauses and checkpoint operations, so its cost was measured alongside its output quality. Total

execution time, LLM calls, retries and tokens were taken from the execution logs. Table 6.10 and Figure 6.3 show the results, with the baseline run as a reference point.

Table 6.10: Execution performance

Run	Total time (min)	LLM calls	Retries	Tokens
Item Listing	36.4	24	4	31,800
Authentication	52.7	31	6	42,600
Shopping Cart	56.8	34	7	47,900
Baseline	31.2	18	3	25,400

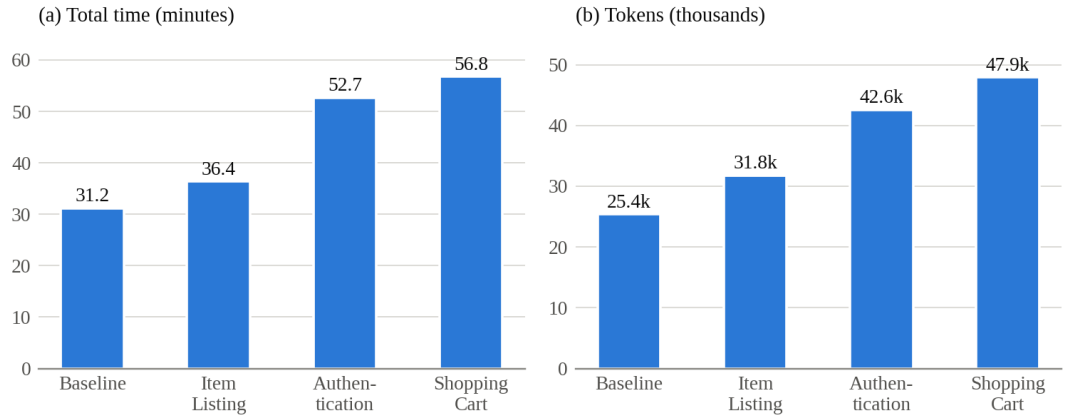


Figure 6.3: Execution time and token usage per run

Cost rose with complexity. Compared with the baseline run, Item Listing took 5.2 minutes longer and used six more LLM calls, while Shopping Cart took 25.6 minutes longer, used 16 more calls and nearly twice the tokens. Retries also rose with complexity, from three to seven, which reflects the agents' re-prompting and repair loops doing their job on harder features. None of these runs is fast, and a team adopting AutoForge would be trading time for reviewability and fewer defects. The figures make that trade explicit.

6.4 Stakeholder and user validation

The validation reported in this chapter comes from the human decisions recorded inside the pipeline. At every stage of every run, a reviewer read the artefact and approved it, rejected it or asked for a revision, and the system recorded the decision and the resulting version. These decisions are the source of the revision counts, the human-gates comparison and the 36-minute review effort figure. They show that reviewers could understand the artefacts well enough to find real problems in them,

such as the missing Admin and Manager user stories and the unnecessary identifier field.

Formal, structured validation with external stakeholders has not yet been done. We did not run interviews, questionnaires or a usability study with practising developers, business analysts or store owners, and the reviews in the evaluation runs were not carried out by independent practitioners. As a result, we can say that the artefacts were reviewable and that review improved the outcome, but not yet how satisfied real users would be, how long reviews would take for someone unfamiliar with the system, or which stages users would choose to gate. These questions are the first priority for further work.

We also collected informal written feedback from five practising professionals: three software engineers, a business analyst and a QA technical lead. The response was encouraging. The engineers said the pipeline's roles match how real teams work, and that the live preview and generated artefacts made it easy to follow. The business analyst found the SRS well structured and called her suggestions refinements, not major problems. The QA lead rated the QA design positively and liked that it treats test results as decision support, not proof. The feedback was unstructured, so it is not a usability study, but it suggests the design is understandable and realistic.

The reviewers' suggestions were constructive and point to clear next steps. Several asked for risk-based approval gates, so that low-risk changes could pass automatically while serious ones still need a human, which would cut review effort. Others suggested more domains and tech stacks beyond e-commerce on Next.js, Express and MongoDB, to show the approach works more widely. The business analyst asked for clearer user roles and measurable non-functional requirements, and the QA lead asked for requirement-to-test traceability and mutation testing to show the generated tests are effective. Further supporting evidence is provided in Appendix D.

6.5 Achievement of common objectives

Table 6.11 maps the evidence to each group objective from Section 2.4 and states whether it was fully or partly achieved.

Table 6.11: Achievement of group objectives

ID	Status	Justification
GO1	Achieved	All seven stages produced a versioned artefact with an explicit human decision for Item Listing (Table 6.3), and the pipeline completed for all three features
GO2	Achieved	7/7 controlled recovery trials resumed correctly, and one real backend failure recovered from its checkpoint (Table 6.9)
GO3	Achieved	Retrieval reduced missing requirements from 7 to 2 and revisions from 6 to 3 (Table 6.5); shown on one controlled request
GO4	Achieved	Architecture and UI/UX artefacts were checked automatically and approved after revision (v3 each); the fixes were caught before coding
GO5	Achieved	The Coder change was built in the sandbox and merged only after approval at v4 (Table 6.3)
GO6	Partly achieved	Precision 87.5%, recall 77.8% and coverage 79% were measured (Tables 6.6 and 6.7), but on one vulnerability set, and QA cannot yet test route handlers or interface components
GO7	Partly achieved	Three features of increasing complexity completed and review cost was measured (Tables 6.4, 6.8 and 6.10), but three features are not enough for benchmark-style claims

Overall, the objectives that concern the design of the pipeline (control, durability, grounding and verification) were met. The objectives that depend on scale of evaluation were met only partly, because the evidence comes from a small number of features and one vulnerability set.

6.6 Integrated limitations and threats

The results should be read with several limitations in mind. The most important is scale. The evaluation covers three features in one domain with one default model. That is enough to show that the pipeline works end to end and to measure the effect of the main design choices, but not enough to estimate defect rates or false-positive rates with statistical confidence across many applications.

There are also dependencies between components that limit what each result can show. The QA Agent's results depend on the code the Coder Agent produced and on the sandbox it built, and the Coder Agent's output depends on the quality of the approved design. A weakness early in the pipeline can therefore show up as a test

failure much later. The human gates are designed to catch this, but they also mean that the result for any one stage is partly a result of the reviewers' choices at earlier stages.

Threats to internal validity include the fact that the Item Listing specification and architecture went through several review rounds before the reported versions, so later stages were built on artefacts that had already matured. The sandboxed build and test steps reduce this threat for the Coder, Security and QA stages, but the earlier stages rest on human judgement rather than an independent oracle. Reviewer familiarity with the system may also have affected the revision counts and the 36-minute review time.

Threats to external validity follow from the scope: one domain, one technology stack for the generated application, and one default model. We have not tested AutoForge on other domains, other stacks or other models. Threats to construct validity include the use of passing builds and tests as a proxy for correctness, which they are not, and the fact that the QA Agent's fallback generator recognises only two code shapes, so a module that matches neither gets no fallback test. This understates rather than overstates coverage.

Finally, LLM output is not fully deterministic. The same request can produce a slightly different artefact on another run, and while the schemas, checks and human gates contain this variation, they do not remove it. Repeating each experiment several times and reporting the spread would strengthen the results, and this is part of the future work set out in Chapter 8.

7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION

7.1 Value proposition and beneficiaries

The commercial case for AutoForge rests on one idea: teams that build software with AI need to be able to trust and audit what the AI produced before it ships. Most AI development tools compete on how quickly they produce a finished result. AutoForge competes on how much of the process a team can see and control, and on evidence that this control improves the result.

There is clear evidence that this need is real. In the 2025 Stack Overflow Developer Survey, 84% of developers said they use or plan to use AI tools, but only 32.7% trusted the accuracy of the output, while 45.7% distrusted it. The most common frustration, named by 66% of respondents, was AI output that is "almost right, but not quite" [23]. Developers are adopting AI faster than they are learning to trust it. A pipeline that shows every intermediate decision, and lets a person correct it before it travels further, is aimed directly at that gap.

Our own evaluation gives a buyer measured evidence rather than a promise. With the human gates switched on, escaped defects fell from five to one and the QA pass rate rose from 78% to 91%, for 36 minutes of review effort (Table 6.8). Retrieval-based enrichment reduced missing requirements from seven to two (Table 6.5), the Security Agent reached 87.5% precision and 77.8% recall on injected vulnerabilities (Table 6.6), and every controlled interruption resumed at the correct stage (Table 6.9). Table 7.1 shows how this value reaches each group of beneficiaries.

Table 7.1: Value created for each beneficiary

Beneficiary	Need	What AutoForge provides	Supporting evidence
Small development teams and agencies	Deliver client features faster without losing control or quality	A seven-stage pipeline with a reviewed, versioned artefact at every stage	Gates ON: 1 escaped defect instead of 5 (Table 6.8)

Beneficiary	Need	What AutoForge provides	Supporting evidence
Reviewers (analyst, architect, designer, QA and security)	See and steer each decision in their own area	A gate at their stage, with diagrams, previews, diffs and findings to review	4 of 7 stages revised on the first feature (Table 6.3)
Store owners (the clients)	A store that works and holds customer data safely	Scanned and tested code, and an approval trail they can inspect	QA pass rate 91% with gates ON (Table 6.8)
Store customers	Safe accounts, carts and orders	Every change scanned for exposed credentials, unsafe code and vulnerable dependencies	Security precision 87.5%, recall 77.8% (Table 6.6)
Universities and students	Teach the full software life cycle with modern AI tools	A free, self-hosted edition that makes every stage visible	Runs on a single machine with a local model (Section 5.3)

The value proposition can be summarized in four parts. AutoForge offers control, because no stage advances without a human decision; durability, because a run can wait for a reviewer and survive a restart; measured trust, because the security and QA results are measured rather than claimed; and low running cost, because the default model runs locally without per-token fees.

The evidence above comes from our own controlled study. To confirm demand from the people who would buy the product, we are collecting three further kinds of stakeholder evidence: a survey of developers, analysts and agency staff, letters of

interest from development agencies, and feedback from demonstration sessions. Table 7.2 records the results.

Table 7.2: Stakeholder evidence of demand

Evidence	Participants	Key result
Online survey (10 questions)	15 (developers, analysts, QA engineers)	11 of 15 (73%) would use it; 6 of 15 (40%) willing to pay LKR 6,000 or more; median delivery time 3 days
Demonstration sessions	10 practitioners	Mean rating 4.1 out of 5 (n = 10); main comments: approval gates make AI output easier to trust, review time is a concern, and a cheaper tier is wanted

7.2 Market alternatives and differentiation

Our first target segment is small e-commerce development teams and digital agencies that build online stores for clients. We plan to begin in Sri Lanka, where the IT and business process management industry employs about 175,000 people across more than 680 companies and over 700 active startups, and earned an estimated USD 2 billion in exports in 2025 [24]. Many of these firms are small and do client work, which is exactly the setting in which a specification, a design record and an approval trail have commercial value. Two further segments follow: in-house teams at retailers that build their own stores, and universities that teach software engineering. Once domain knowledge packs exist for other bounded domains, teams outside e-commerce become a third segment.

Buyers already pay for AI development tools, and the market has settled on a common way of charging. Table 7.3 lists the current list prices of the most widely used tools. Every coding tool combines a price per seat with a usage allowance, and team plans cost between USD 25 and USD 50 per user per month, or about LKR

8,300 to 16,500 at the October 2026 exchange rate of 330.43 rupees to the dollar [33].

Table 7.3: Pricing existing AI development tools (October 2026)

Product	Type	Individual plan (USD/month)	Team plan (USD/user/month)	Usage limited by
GitHub Copilot [26]	Coding assistant	Pro 10; Pro+ 39	Business (price not listed); Enterprise on request	AI credits
Cursor [27]	AI code editor	20	Teams 40	Agent usage
Lovable [28]	Prompt-to-app builder	Pro from 25	Business from 50	Credits
Bolt [29]	Prompt-to-app builder	Pro 25	Teams 30	Tokens
ChatGPT [30]	Chat assistant	Go 8; Plus 20	Business 25	Messages
Claude [31]	Chat assistant	Pro 20	Team 25 (standard seat)	Usage limits
Gemini [32]	Chat assistant	AI Plus 4.99; AI Pro 19.99	Not listed	Usage limits

These tools fall into five groups of alternatives to AutoForge. Prompt-to-app builders such as Lovable and Bolt turn a description into a working application in one step, but leave no intermediate artefacts to review, which is the problem this project starts from. Coding assistants such as GitHub Copilot and Cursor make individual developers faster, but they work inside the code and do not produce a specification, an architecture or a test report. Issue-resolution agents such as SWE-agent [5], OpenHands [6] and Agentless [9] work well on existing repositories but assume that

the task has already been specified. Role-based multi-agent frameworks such as ChatDev [2] and MetaGPT [3] cover more of the life cycle but review through agent dialogue and do not offer durable human checkpoints. Human-in-the-loop assistants such as HULA [13] confirm that engineers want oversight at the planning stage, but work inside one host tool. Table 7.4 compares them.

Table 7.4: Alternatives and AutoForge's differentiation

Alternative	Strength	Gap that AutoForge addresses
Prompt-to-app builders (Lovable, Bolt)	Very fast first result	No specification, design or diff to review; hard to locate errors
Coding assistants (GitHub Copilot, Cursor)	Make individual developers faster	Work at the level of code only; no requirements, design, security or QA stages
Issue-resolution agents (SWE-agent, OpenHands)	Strong on existing codebases	Start from a specified task; no requirements, architecture or UI stages
Role-based multi-agent frameworks (ChatDev, MetaGPT)	Cover several SDLC phases	Review through agent dialogue; runs lost on interruption; no versioned stage artefacts
Human-in-the-loop assistants (HULA)	Fit current workflows	Oversight limited to the host tool rather than a full gated pipeline
AutoForge	**Gate at every stage; versioned artefacts; measured security and QA; local model**	**Currently one domain and one generated stack**

The differentiation that is hardest to copy is not any single feature but the combination, backed by evaluation data. Adding an approval button to an existing generator does not give it durable checkpoints, retrieval-grounded specifications or

executable verification, and our results show that each of these contributes something measurable. Two assets also build up over time: the domain knowledge packs, which take expertise to write, and the evaluation method, which lets us show a customer measured results on their own work.

7.3 Business model and feasibility

We propose an open-core model, which suits both the local-first design and a student team's resources. The core pipeline is free and self-hosted, and customers pay for what a business needs on top of it. We considered copying the plans of chat assistants such as ChatGPT, Claude and Gemini, which limit the number of messages or queries. That model does not fit AutoForge for two reasons. First, the default model runs on the customer's own machine, so a query costs us nothing to serve and there is nothing to ration. Second, a customer does not value a message; they value a reviewed and tested feature. We therefore follow the coding tools in Table 7.3 and charge for seats and usage, but measure usage in feature runs. A feature run is one feature taken through all seven stages, from requirement to QA report, including revisions.

Table 7.5 sets out the proposed packages. The paid seats are priced just below the LKR 8,260 that teams already pay for ChatGPT Business or Claude Team, and well below Cursor Teams and Lovable Business.

Table 7.5: Proposed AutoForge packages

Package	Price (LKR)	Target users	Main features	Feature runs
Community	Free	Students and individual developers	Self-hosted; local model through Ollama; all seven agents; e-commerce knowledge base; one reviewer	Unlimited on own hardware

Package	Price (LKR)	Target users	Main features	Feature runs
Team	6,000 per seat per month (minimum 3 seats; 5,000 if paid yearly)	Small development teams	Hosted pipeline; several reviewers with roles; audit log; version history and revoke	15 per seat, pooled
Agency	10,000 per seat per month (minimum 3 seats)	Agencies building stores for clients	Team features, plus a workspace and design system per client, client sign-off of page previews, one extra knowledge pack and priority support	30 per seat, pooled
Enterprise / On-premise	From 1,500,000 per year	Larger firms that keep code in-house	Installed in the customer's network; single sign-on; custom knowledge packs; support agreement	Unlimited on own hardware
Academic	Free for students; 150,000 per year for a hosted campus lab	Universities	Community features on a shared lab server for up to 100 students	Fair use

Three further revenue streams sit alongside the packages: extra hosted feature runs at LKR 400 each, domain knowledge packs for other bounded domains such as learning management or hotel booking at LKR 30,000 each, and setup and training services at

LKR 75,000 per engagement for early customers. Table 7.6 summarises the business model.

Table 7.6: Business model summary

Element	Proposal
Customer segments	Small e-commerce development teams; agencies building stores for clients; universities; later, teams in other bounded domains
Channels	Open-source Community edition on GitHub; SLASSCOM and industry events; university partnerships; direct sales to agencies
Key partners	Pilot agencies; open-source projects the system depends on; model providers for the hosted option; SLIIT
Key resources	The pipeline and its agents; domain knowledge packs; evaluation method and data
Cost structure	Development and maintenance; hosting and model fees for the hosted option; knowledge pack curation; support
Revenue streams	Team and Agency subscriptions; enterprise and academic licences; extra feature runs; knowledge packs; setup and training

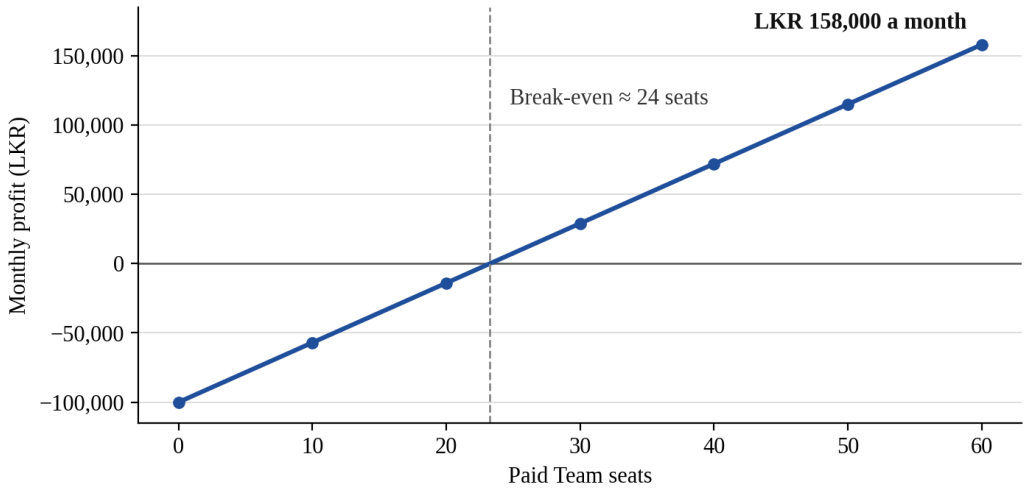
Table 7.7 shows the one-time investment made to develop the current system. Human effort accounts for 62.2% of it; the software stack itself is open source.

Table 7.7: One-time development investment

Category	Item	Cost (LKR)
Hardware	RAM upgrade for the development environment	45,000
Hardware	Shared GPU resource for the local model	59,750

Category	Item	Cost (LKR)
Software	Software licences and tools	20,000
Cloud services	Cloud hosting and CI/CD	15,000
API usage	OpenAI and Anthropic API usage during testing	6,000
Human effort	System design, implementation, testing and documentation	240,000
Total		**385,750**

Running a hosted service would add monthly costs. We estimate fixed costs of about LKR 100,000 a month: LKR 25,000 for a cloud server running the API, MongoDB and the Docker sandbox; LKR 60,000 for part-time maintenance and support; LKR 10,000 for marketing; and LKR 5,000 for other services. The variable cost of a Team seat is about LKR 1,700 a month, covering model fees, sandbox compute and payment fees, so each Team seat contributes about LKR 4,300. Figure 7.1 shows that the hosted service breaks even at about 24 Team seats, the equivalent of eight small teams of three. At 40 seats it earns about LKR 72,000 a month above its running costs, which recovers the development investment in about five and a half months.



Assumes LKR 6,000 per seat, LKR 1,700 variable cost per seat and LKR 100,000 fixed costs a month.

Figure 7.1: Monthly profit of the hosted service by number of Team seats

The technical feasibility of the product is supported by the evaluation itself: the complete pipeline ran end to end on three features of increasing complexity on a single local machine. The cost of each run is also known. Each run used between 25,400 and 47,900 tokens (Table 6.10). With the default local model these tokens carry no API fee. On a hosted model, even if every token were billed at the output price of a current mid-range model, USD 10 per million tokens [31], the largest run would cost about USD 0.48, or LKR 160. A realistic mix of input and output tokens costs about LKR 70. Pricing the hosted option is therefore a calculation rather than a guess.

Operational feasibility is the harder question. A run currently takes between about half an hour and an hour (Table 6.10), plus review time. For a team building a store this is acceptable for a feature that would otherwise take days, but not for quick, interactive edits. The current single-machine deployment would also need to become a multi-user service before it could support paying teams. That means user accounts with separate data for each customer, a job queue so that several runs can happen at once, and one sandbox per run. The cost of the service then grows with the number of feature runs rather than the number of customers.

We will validate the business case in stages before scaling, as Table 7.8 shows. Each phase has a test that it must pass before the next begins. The most important is the

design-partner pilot, in which one small agency uses AutoForge on a real client project and we record the same measures as in Chapter 6 on their work instead of ours.

Table 7.8: Validation and launch plan

Phase	Period	Activity	Test to pass
1. Evidence	To December 2026	Survey, letters of interest and demonstration sessions (Table 7.2)	60% would use a gated pipeline; at least one letter of interest
2. Pilot	January to March 2027	One agency uses AutoForge free on a client feature; revisions, defects, QA pass rate and review time recorded	At least one developer-day saved per feature; agency willing to pay
3. Launch	April to June 2027	Community edition published on GitHub; hosted Team beta opened	10 paying seats
4. Growth	July to December 2027	Agency tier, first paid knowledge pack, outreach to agencies in South Asia	24 seats (break-even)

7.4 Adoption intellectual property and compliance

The main barriers to adoption are trust, time and hardware. Teams that have been disappointed by AI tools may be sceptical; the measured security and QA results are our best answer to this, but a pilot with real users will matter more. Review time is a real cost, and some teams may prefer to gate only the stages they consider critical, which is a direction our discussion of the results already suggests. Running a capable model locally also needs reasonably powerful hardware, which the hosted Team and Agency packages address.

On intellectual property, the code a customer generates with AutoForge belongs to the customer, and the licence terms will say so clearly. The product itself is built on open-source components and on an openly available model, so a licence review of every dependency, and of the model's terms of use, is needed before any commercial release. The domain knowledge packs are written by the team and would remain our property. Ownership of the AutoForge code developed in this project is also subject to the university's intellectual property policy, which would have to be settled before a commercial venture could begin.

Data protection matters at two levels. The stores that AutoForge generates hold personal data such as accounts, addresses and orders. In Sri Lanka they fall under the Personal Data Protection Act, No. 9 of 2022, whose main obligations take effect on 1 January 2027 [25]. Store operators must then process personal data lawfully and transparently, report breaches, carry out impact assessments where required and follow rules on transfers abroad. AutoForge's Security Agent helps by catching exposed credentials and unsafe code, but it does not by itself make a store compliant; that remains the responsibility of the team that builds and runs it. At the level of the tool, the self-hosted editions keep a customer's requirements and code on their own network. The hosted service would itself be a data controller for its users' data, and would have to state clearly which model provider, if any, receives a customer's prompts.

There are also social, ethical and professional aspects to consider. The first is accountability: because every stage has an explicit approval, there is always a named person who accepted each artefact, which is not the case when an AI tool produces finished code directly. The second is transparency: the pipeline records which version was approved, and the QA Agent even records whether each test came from the model or from a template. The third is automation bias, the risk that reviewers start approving whatever the agents produce. Showing reviewers the real git diff, rendered previews and measured findings is meant to make review meaningful, but the risk does not disappear, and training reviewers matters. The last is the effect on work. AutoForge is designed to support developers, not replace them, because a person stays responsible at every stage. Even so, tools like this may reduce some

routine development tasks, and that is a real social consideration rather than something to dismiss.

7.5 Sustainable Development Goal contribution

Section 1.3 linked the project to SDG 9 (Industry, Innovation and Infrastructure) and SDG 8 (Decent Work and Economic Growth). The plausible contribution to SDG 9 relates to target 9.5, enhancing research and upgrading technological capabilities: AutoForge is a tested, open method for using AI in software engineering under human control, and its free self-hosted edition lets local firms and universities adopt it without depending on expensive external services. The plausible contribution to SDG 8 relates to targets 8.2 and 8.3, higher productivity through technological upgrading and the growth of small and medium-sized enterprises: small development firms could deliver reviewed, tested features faster, and through them small businesses could get more affordable online stores.

Both contributions are potential rather than demonstrated. AutoForge has not yet been deployed with real businesses, so we cannot report actual effects on productivity or growth; the pilot in Table 7.8 is designed to measure them. There are also possible negative effects. Running large models locally uses energy, and wider use could add to it. Reducing routine coding work could reduce opportunities for junior developers to learn by doing that work. To sustain the benefits and limit the harms, we would measure energy and time per feature in every pilot, keep the human review stages as a place where junior developers learn by reviewing and correcting real artefacts, offer the Academic package so that students learn to work with AI under supervision, and continue to publish evaluation results so that claims about the tool can be checked.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Overall conclusion

This project set out to answer a practical question: can LLM agents carry a web application feature through the whole development life cycle in a way that keeps a person in control at every step, survives interruption, grounds the specification in domain knowledge and ends with verified rather than claimed results? For the e-commerce features we evaluated, the answer is yes.

AutoForge carried three features of increasing complexity through all seven stages. Every stage produced a versioned artefact and an explicit human decision, and on the first feature, reviewers found and fixed real problems at four of the seven stages before they could spread. The evaluation also showed that each of the main design choices does measurable work. Retrieval made the specification more complete. Human gates raised the QA pass rate and cut escaped defects from five to one. Checkpointing recovered the pipeline in every controlled trial and after a real failure. The security and QA stages produced findings and test results that could be checked against known outcomes.

The answer comes with a cost, and we have tried to report it honestly. The gated pipeline takes longer and uses more model calls than a baseline run, and human review took 36 minutes that a fully automatic pipeline would not need. The evidence also comes from a small number of features in one domain. Within those limits, the results support a checkpointed, human-approved pipeline as a controlled and auditable way to use LLMs in software development.

8.2 Principal integrated contribution

The main contribution of the team is not any single agent. Each component on its own addresses a known problem, and the individual theses describe them in detail. What only exists because the four components were built together is the process: a pipeline in which every intermediate artefact is versioned, reviewed by a person and recoverable after interruption, so that the point where a requirement or implementation decision changed can be seen before the error reaches later stages.

Several things were learned at the level of the whole system that no single component would have shown. Human review is most valuable early: the revisions that mattered most were in the requirements, the architecture and the data model, not in the final reports. Integration works best through artefacts with agreed shapes rather than through agents calling each other. Shared infrastructure, such as the one sandbox used by three agents, makes the evidence more trustworthy because every check runs against exactly the code that was built. And the value of human oversight can, and should, be measured rather than assumed.

8.3 Future directions

We see the following directions as the most important, in order of priority.

1. Generalize beyond e-commerce. Because the Domain Agent's knowledge comes from a retrievable knowledge base, swapping that base for another bounded domain, such as learning management or hospital management, is a natural next step.
2. Strengthen individual agents. The QA Agent should be able to test route handlers and interface components, which needs a DOM renderer or a running application in the sandbox. The Coder Agent could retrieve code from the same repository to improve generation, in the way RepoCoder [12] does.
3. Add an integrated code editor to the review interface, so reviewers can correct generated code directly at the gate instead of only approving, rejecting or requesting regeneration.

REFERENCES

[1] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–79, 2024.

[2] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, *et al.*, "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, 2024, pp. 15174–15186.

[3] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, S. Yau, Z. Lin, L. Zhou, *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024, pp. 23247–23275.

[4] J. He, C. Treude, and D. Lo, "LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–30, 2025.

[5] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 50528–50652.

[6] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, *et al.*, "OpenHands: An open platform for AI software developers as generalist agents," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2025, pp. 65882–65919.

[7] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, "Executable code actions elicit better LLM agents," *arXiv preprint* arXiv:2402.01030, 2024.

[8] H. N. Phan, T. N. Nguyen, P. X. Nguyen, and N. D. Bui, "HyperAgent: Generalist software engineering agents to solve coding tasks at scale," *arXiv preprint* arXiv:2409.16299, 2024.

[9] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," *arXiv preprint* arXiv:2407.01489, 2024.

[10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024, pp. 54107–54157.

[11] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," *arXiv preprint* arXiv:2312.13010, 2023.

[12] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "RepoCoder: Repository-level code completion through iterative retrieval and generation," in *Proc. 2023 Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2023, pp. 2471–2484.

[13] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang, J. Li, E. Cook, K. Chen, and M. Wu, "Human-in-the-loop software development agents," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*, 2025, pp. 342–352.

[14] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[15] A. Hemmat, M. Sharbaf, S. Kolahdouz-Rahimi, K. Lano, and S. Y. Tehrani, "Research directions for using LLM in software requirement engineering: A systematic review," *Frontiers in Computer Science*, vol. 7, Art. no. 1519437, 2025.

[16] O. C. Z. Gotel and A. C. W. Finkelstein, "An analysis of the requirements traceability problem," in *Proc. IEEE Int. Conf. Requirements Engineering*, 1994, pp. 94–101.

[17] J. Mucha, A. Kaufmann, and D. Riehle, "A systematic literature review of pre-requirements specification traceability," *Requirements Engineering*, vol. 29, no. 2, pp. 119–141, 2024.

[18] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Trans. Softw. Eng.*, vol. 50, no. 4, pp. 911–936, 2024.

[19] X. Zhu, W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang, "When software security meets large language models: A survey," *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 2, pp. 317–334, 2025.

[20] K. Pelluru, "LangChain & LangGraph in production: Architectures for multi-agent LLM systems," *Journal of Data and Digital Innovation (JDDI)*, vol. 2, no. 3, pp. 1–9, 2025.

[21] Chroma, "Chroma documentation." [Online]. Available: https://docs.trychroma.com/ (accessed Aug. 25, 2026).

[22] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[23] Stack Overflow, “2025 Developer Survey: AI,” 2025. [Online]. Available: https://survey.stackoverflow.co/2025/ai (accessed Oct. 8, 2026).

[24] “SLASSCOM reflects on a year of creating opportunities, strengthening Sri Lanka’s knowledge and innovation industry,” Daily FT, Jul. 15, 2026. [Online]. Available: https://www.ft.lk/it-telecom-tech/SLASSCOM-reflects-on-a-year-of-creating-opportunities-strengthening-Sri-Lanka-s-knowledge-and-innovation-industry/50-794672 (accessed Oct. 8, 2026).

[25] “Data protection compliance regime takes effect on 1 Jan. 2027,” Daily FT, 2026. [Online]. Available: https://www.ft.lk/front-page/Data-protection-compliance-regime-takes-effect-on-1-Jan-2027/44-795776 (accessed Oct. 8, 2026).

[26] GitHub, “GitHub Copilot plans and pricing.” [Online]. Available: https://github.com/features/copilot/plans (accessed Oct. 8, 2026).

[27] Anysphere, “Cursor pricing.” [Online]. Available: https://cursor.com/pricing (accessed Oct. 8, 2026).

[28] Lovable, “Subscription plans.” [Online]. Available: https://docs.lovable.dev/introduction/subscription-plans (accessed Oct. 8, 2026).

[29] StackBlitz, “Bolt pricing.” [Online]. Available: https://bolt.new/pricing (accessed Oct. 8, 2026).

[30] “ChatGPT pricing in 2026: Free, Go, Plus, Pro, Business and Enterprise,” GPTPrompts.ai. [Online]. Available: https://gptprompts.ai/ai-pricing/chatgpt-pricing (accessed Oct. 8, 2026).

[31] Anthropic, “Claude pricing.” [Online]. Available: https://claude.com/pricing (accessed Oct. 8, 2026).

[32] Google, “Google AI subscriptions.” [Online]. Available: https://gemini.google/subscriptions/ (accessed Oct. 8, 2026).

[33] Exchange-Rates.org, “USD to LKR: Convert US Dollars to Sri Lanka Rupees.” [Online]. Available: https://www.exchange-rates.org/converter/usd-lkr (accessed Oct. 7, 2026).

APPENDICES

Appendix A Integrated interface specification

This appendix specifies the interfaces that join the four components. It describes the information each interface carries and the rules that apply to it; the field-level schemas are documented with each agent's package and in the individual theses.

A.1 Stage gate interface

Table A.1: Stage gate interface

Element	Specification
Scope	Identical for all seven stages: Requirement, Domain, Architecture, UI/UX, Coder, Security, QA
Trigger	Agent finishes generating and checking its artefact; state is checkpointed; graph pauses via interrupt()
Information shown to reviewer	Current artefact version (Markdown), with diagrams, previews, diff or findings as applicable
Decisions accepted	Approve, reject, or request revision (with the reviewer's comments)
Approve	Records the decision and resumes the graph at the next stage
Request revision	Records the decision; the same agent produces version N+1; earlier versions are retained
Reject	Records the decision; the output is discarded (for the Coder stage, the feature branch is dropped)
Persistence	Decision and pipeline position are saved together in the MongoDB checkpoint

A.2 Shared sandbox interface

The Docker sandbox created for a code change is passed to the Security and QA agents as a dependency-injected parameter. Through it, the Security Agent runs npm audit against the sandbox's lockfile and the QA Agent runs the generated tests, so both report on exactly the code that was built and shown to the reviewer.

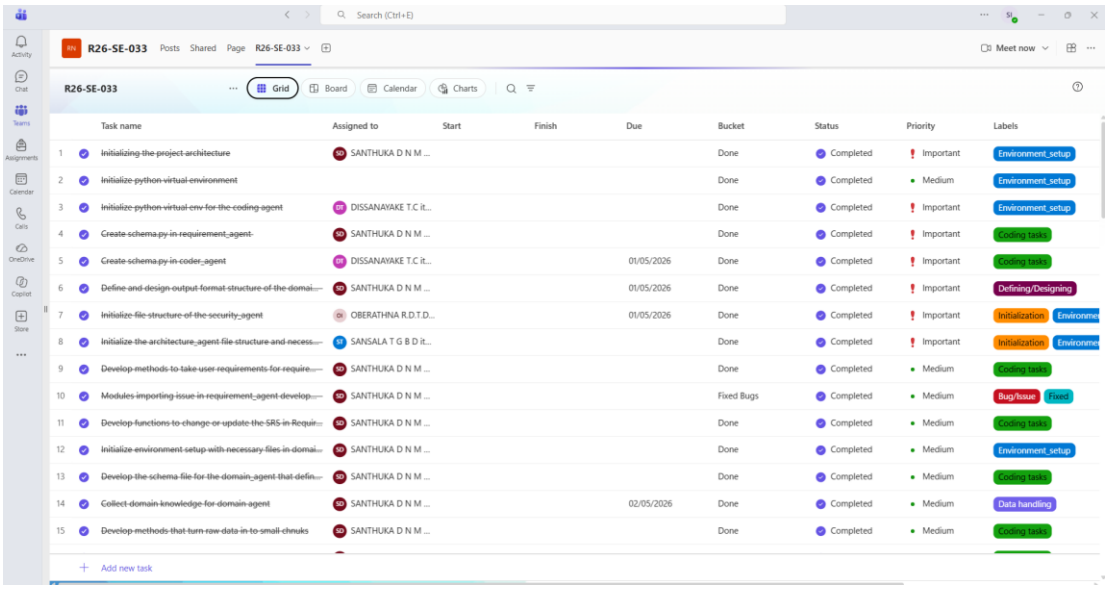
Appendix B Team contribution declaration

The following record of individual and shared contributions has been agreed by all four members.

Table B.1: Individual and shared contributions

Member	Individual contribution	Shared contribution
Santhuka D.N.M.D. (IT22098078)	Requirement Agent; Domain Agent and knowledge base; LangGraph orchestration, gates and MongoDB checkpointing	Integration of all stages; evaluation; research paper; this report
Sansala T.G.B.D. (IT22174826)	Architecture Agent with UML generation and consistency checks; UI/UX Agent with design system and previews	Evaluation; research paper; this report
Dissanayake T.C. (IT22157232)	Coder Agent with branch isolation and sandboxed build; review interface pages	Shared sandbox; evaluation; research paper; this report
Oberathna R.D.T.D. (IT22102478)	Security Agent with AST, secret and dependency scanning and severity gate; QA Agent with test generation and execution	Security and QA evaluation; research paper; this report

Appendix C Microsoft Teams Planner



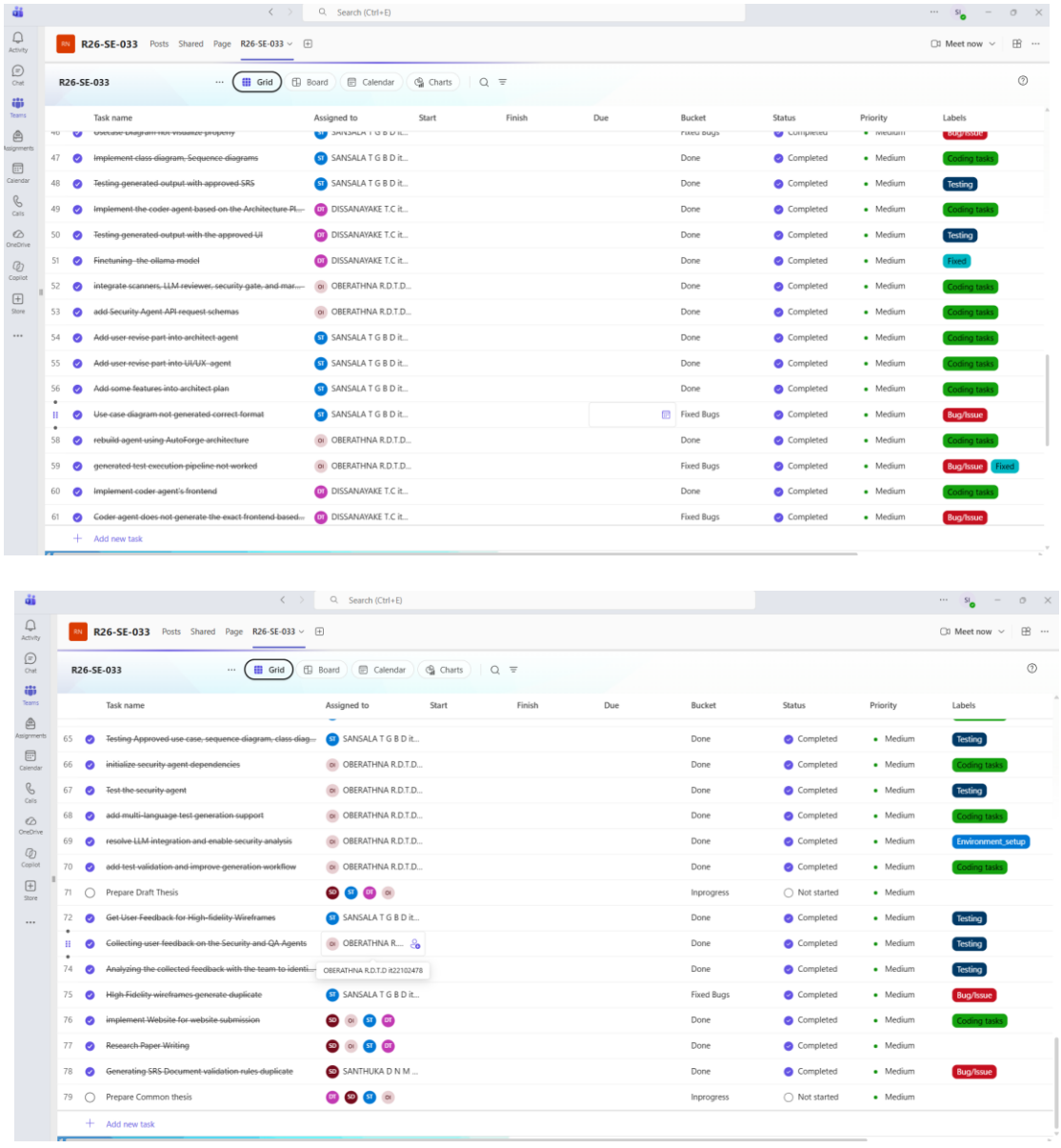
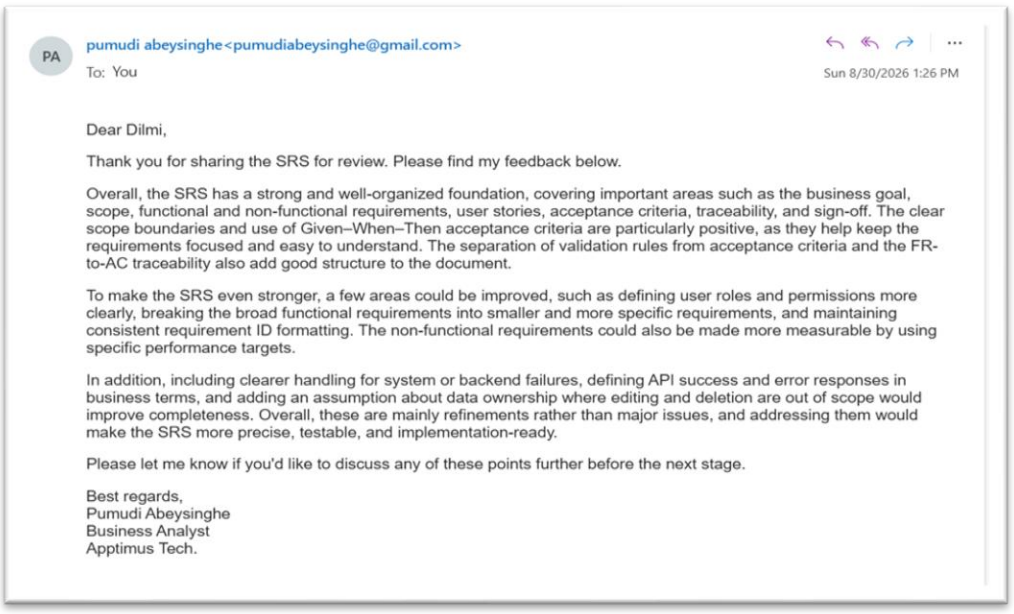


Figure C.1: Microsoft Teams Planner

Appendix D User Evidence

- **Business Analyst Feedback**



PA

pumudi abeysinghe<pumudiabeysinghe@gmail.com>

To: You

Sun 8/30/2026 1:26 PM

Dear Dilmi,

Thank you for sharing the SRS for review. Please find my feedback below.

Overall, the SRS has a strong and well-organized foundation, covering important areas such as the business goal, scope, functional and non-functional requirements, user stories, acceptance criteria, traceability, and sign-off. The clear scope boundaries and use of Given–When–Then acceptance criteria are particularly positive, as they help keep the requirements focused and easy to understand. The separation of validation rules from acceptance criteria and the FR-to-AC traceability also add good structure to the document.

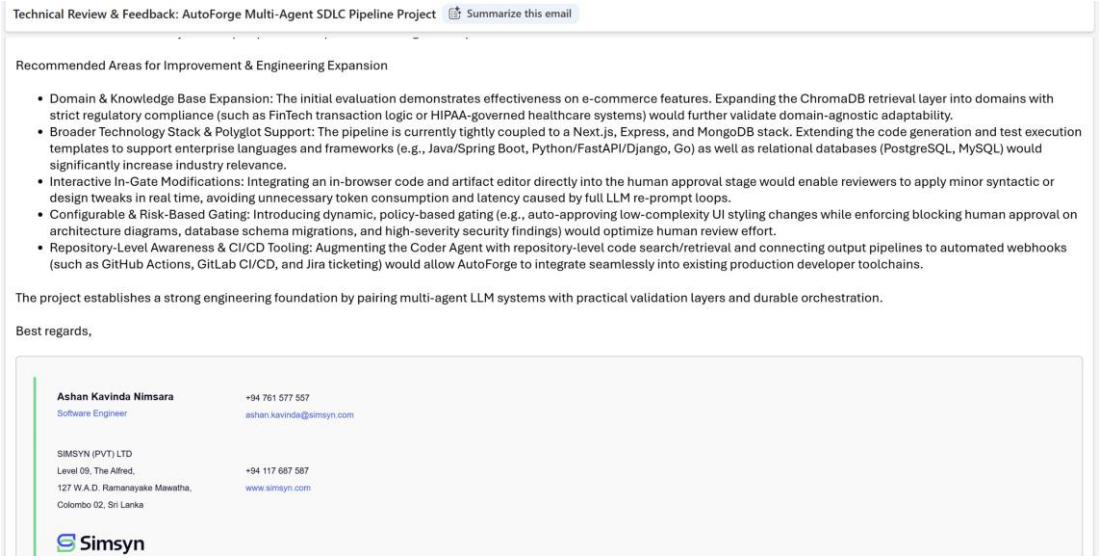
To make the SRS even stronger, a few areas could be improved, such as defining user roles and permissions more clearly, breaking the broad functional requirements into smaller and more specific requirements, and maintaining consistent requirement ID formatting. The non-functional requirements could also be made more measurable by using specific performance targets.

In addition, including clearer handling for system or backend failures, defining API success and error responses in business terms, and adding an assumption about data ownership where editing and deletion are out of scope would improve completeness. Overall, these are mainly refinements rather than major issues, and addressing them would make the SRS more precise, testable, and implementation-ready.

Please let me know if you'd like to discuss any of these points further before the next stage.

Best regards,
Pumudi Abeysinghe
Business Analyst
Apptimus Tech.

- **Overall Feedback**



Technical Review & Feedback: AutoForge Multi-Agent SDLC Pipeline Project Summarize this email

Recommended Areas for Improvement & Engineering Expansion

- Domain & Knowledge Base Expansion: The initial evaluation demonstrates effectiveness on e-commerce features. Expanding the ChromaDB retrieval layer into domains with strict regulatory compliance (such as FinTech transaction logic or HIPAA-governed healthcare systems) would further validate domain-agnostic adaptability.
- Broader Technology Stack & Polyglot Support: The pipeline is currently tightly coupled to a Next.js, Express, and MongoDB stack. Extending the code generation and test execution templates to support enterprise languages and frameworks (e.g., Java/Spring Boot, Python/FastAPI/Django, Go) as well as relational databases (PostgreSQL, MySQL) would significantly increase industry relevance.
- Interactive In-Gate Modifications: Integrating an in-browser code and artifact editor directly into the human approval stage would enable reviewers to apply minor syntactic or design tweaks in real time, avoiding unnecessary token consumption and latency caused by full LLM re-prompt loops.
- Configurable & Risk-Based Gating: Introducing dynamic, policy-based gating (e.g., auto-approving low-complexity UI styling changes while enforcing blocking human approval on architecture diagrams, database schema migrations, and high-severity security findings) would optimize human review effort.
- Repository-Level Awareness & CI/CD Tooling: Augmenting the Coder Agent with repository-level code search/retrieval and connecting output pipelines to automated webhooks (such as GitHub Actions, GitLab CI/CD, and Jira ticketing) would allow AutoForge to integrate seamlessly into existing production developer toolchains.

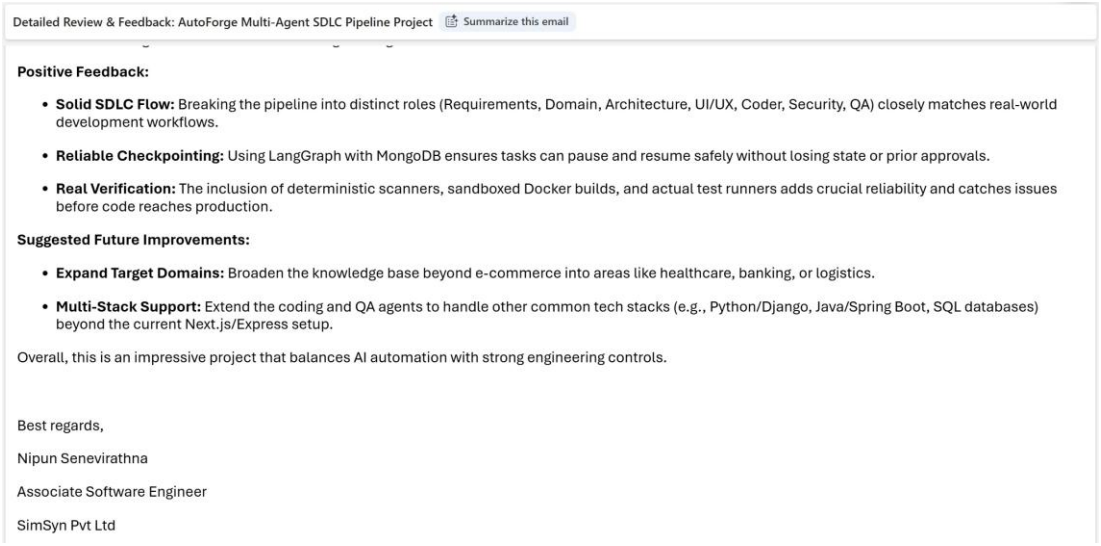
The project establishes a strong engineering foundation by pairing multi-agent LLM systems with practical validation layers and durable orchestration.

Best regards,

Ashan Kavinda Nimsara
Software Engineer
+94 761 577 557
ashan.kavinda@simsyn.com

SIMSYN (PVT) LTD
Level 09, The Alfred,
127 W.A.D. Ramanayake Mawatha,
Colombo 02, Sri Lanka
+94 117 687 587
www.simsyn.com

Simsyn



Detailed Review & Feedback: AutoForge Multi-Agent SDLC Pipeline Project Summarize this email

Positive Feedback:

- **Solid SDLC Flow:** Breaking the pipeline into distinct roles (Requirements, Domain, Architecture, UI/UX, Coder, Security, QA) closely matches real-world development workflows.
- **Reliable Checkpointing:** Using LangGraph with MongoDB ensures tasks can pause and resume safely without losing state or prior approvals.
- **Real Verification:** The inclusion of deterministic scanners, sandboxed Docker builds, and actual test runners adds crucial reliability and catches issues before code reaches production.

Suggested Future Improvements:

- **Expand Target Domains:** Broaden the knowledge base beyond e-commerce into areas like healthcare, banking, or logistics.
- **Multi-Stack Support:** Extend the coding and QA agents to handle other common tech stacks (e.g., Python/Django, Java/Spring Boot, SQL databases) beyond the current Next.js/Express setup.

Overall, this is an impressive project that balances AI automation with strong engineering controls.

Best regards,

Nipun Senevirathna

Associate Software Engineer

SimSyn Pvt Ltd

Figure D.1: Overall technical feedback on the AutoForge system